

# COLLEGE MAJOR ROI DATA PIPELINE

---

An end to end Snowflake, dbt and Airflow pipeline on US college graduate returns

# Contents

---

- 1 Introduction**
- 2 Dataset discussion**
- 3 Methodology**
- 4 Procedure**
  - 4.1 Snowflake
  - 4.2 dbt
  - 4.3 Airflow and Docker
- 5 Insights: the 18 gold tables**
- 6 Discussion: problems and how they were solved**
- 7 Conclusion**
- A Appendices: gold model SQL, gold tests, folder layout, quick reference, housekeeping**

# 1. Introduction

---

This document is a full record of the College Major ROI project: what was built, why, how to rebuild it from nothing, what the data shows, and what went wrong along the way. It is written so that someone who has never seen the project can follow it, recreate it on a Windows machine and run it.

The project takes one CSV file of 30,000 US college graduates and moves it through a complete data pipeline. The file is loaded into Snowflake, cleaned and reshaped by dbt into 18 analysis tables, checked by 49 automated tests, and the whole sequence is run in order by an Airflow workflow that itself runs inside Docker.

There were two goals. The first was practical: to learn how these four tools fit together by building something that runs from start to finish, instead of following separate tutorials. The second was analytical: to use the dataset to look at how much a US degree pays back, and how that changes with the major, the type of school, student debt, internships and region.

## Project at a glance

| Item           | Detail                                                                                                                                                              |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Purpose        | Portfolio project: an orchestrated ELT pipeline, plus an analysis of what US college graduates earn back on their degree.                                           |
| Dataset        | <code>college_major_roi.csv</code> from Kaggle: 30,000 graduates, 18 columns, with a data dictionary.                                                               |
| Warehouse      | Snowflake (Standard edition trial, AWS Asia Pacific Sydney). Database <code>dbt_db</code> , schemas <code>bronze</code> , <code>silver</code> , <code>gold</code> . |
| Transformation | dbt Core 1.12.5 with dbt-snowflake 1.12.1. One seed, one silver view, 18 gold tables, 49 tests.                                                                     |
| Orchestration  | Apache Airflow 2.10.3 running in Docker. One DAG, <code>college_roi_pipeline</code> , with four tasks, LocalExecutor.                                               |
| Local setup    | Windows. Python 3.13. dbt lives in <code>E:\dbt</code> , Airflow lives in <code>E:\airflow</code> .                                                                 |
| Prepared by    | Shaqrان, September 2026. Version 1.0.                                                                                                                               |
| Status         | Runs end to end: all four Airflow tasks pass. Not done: dashboard, scheduled runs, version control.                                                                 |

The rest of the document covers the dataset, the methodology, a step by step procedure split into Snowflake, dbt, and Airflow and Docker, the insights from all 18 gold tables, every problem met along the way with its fix, and a conclusion.

## Headline results

- The pipeline works end to end. One trigger in Airflow validates the file, loads it, builds 19 dbt models and runs 49 tests, and all four tasks pass.
- The major matters more than anything else in the data. STEM graduates averaged \$447,000 net ROI and arts graduates averaged -\$39,821, at almost exactly the same cost of about \$88,000.
- School type matters in a less obvious way. For-profit colleges returned \$102,770 on a cost of \$119,695, while regional colleges returned \$244,433 on a cost of \$65,322.
- Region and GPA moved the averages only a little. Internships and finishing on time were linked with higher returns, but the internship gap sits almost entirely in STEM and business.

## 2. Dataset discussion

### 2.1 Source and scope

The data is a single CSV, `college_major_roi.csv`, downloaded from Kaggle together with a data dictionary, `data_dictionary.csv`. It has 30,000 rows, one per graduate, and 18 columns. It covers 19 majors grouped into 7 major categories, 5 institution tiers and 4 US regions. Every money value is in US dollars and is measured over a 10 year window after graduation.

The idea behind the file is simple. For each graduate it records what the degree cost, what they earned over 10 years, and what they would have earned over the same 10 years if they had stopped at high school. The difference between the two, after subtracting the cost of the degree, is the return.

### 2.2 Column reference

| Column                         | Type    | Meaning                                                                               |
|--------------------------------|---------|---------------------------------------------------------------------------------------|
| grad_id                        | text    | Unique graduate identifier, for example G000000.                                      |
| major                          | text    | Degree major. 19 values, such as biology, nursing or fine_arts.                       |
| major_category                 | text    | Group of majors: STEM, health, business, social_science, education, humanities, arts. |
| institution_tier               | text    | Type of school: ivy_elite, top_public, mid_tier, regional, for_profit.                |
| institution_selectivity_pctile | integer | Admissions selectivity percentile of the school.                                      |
| gpa                            | decimal | Grade point average on a 0 to 4 scale.                                                |
| had_internship                 | integer | 1 if the graduate completed at least one internship, otherwise 0.                     |
| completed_on_time              | integer | 1 if the graduate finished on schedule with no extra years, otherwise 0.              |
| region                         | text    | US region: northeast, south, midwest, west.                                           |
| net_cost_usd                   | integer | Total net cost of the degree after aid.                                               |
| debt_usd                       | integer | Student debt taken on. 0 means none.                                                  |
| hs_baseline_10yr_usd           | integer | Counterfactual 10 year earnings as a high school graduate.                            |
| added_earnings_10yr_usd        | integer | 10 year earnings above the high school baseline: the earnings premium of the degree.  |
| earnings_10yr_usd              | integer | Total 10 year earnings.                                                               |
| net_roi_usd                    | integer | Target. Net ROI = added_earnings_10yr_usd minus net_cost_usd.                         |
| roi_pct                        | decimal | Target. Net ROI as a percentage of net cost.                                          |
| positive_roi                   | integer | Target. 1 if net_roi_usd is above 0.                                                  |
| high_roi                       | integer | Target. 1 if net_roi_usd is in the top quartile.                                      |

### 2.3 How the columns relate

These relationships were checked against the data, not just read from the dictionary:

- `earnings_10yr_usd` equals `hs_baseline_10yr_usd` plus `added_earnings_10yr_usd`, to within \$100.
- `net_roi_usd` equals `added_earnings_10yr_usd` minus `net_cost_usd`, to within \$100. The small gap is explained in section 2.4.
- `roi_pct` is net ROI as a percentage of net cost. Because of the rounding, it can differ from a recalculation by up to about 3 points.
- `positive_roi` is 1 exactly when `net_roi_usd` is above zero. This held for every row.

- **high\_roi** is 1 when net ROI is \$423,525 or more, which is the top quartile. Exactly 25% of rows are flagged.

The dictionary labels the last four columns as targets, which means the file was built with prediction in mind. This project does not build a predictive model. It describes the data.

## 2.4 Data quality findings

| Check                                       | Result                                                                                                                                                    |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Row count                                   | 30,000.                                                                                                                                                   |
| Null values                                 | None in any column. Checked with a null count query on the silver model.                                                                                  |
| Duplicate grad_id                           | None. The unique test on grad_id passes.                                                                                                                  |
| Negative cost, debt or high school baseline | None. The <b>has_negative_value</b> flag is false for every row.                                                                                          |
| Category values outside the dictionary      | None. The accepted_values tests pass for major_category, institution_tier and region.                                                                     |
| Net ROI against added earnings minus cost   | 7,554 rows (25.2%) differ by exactly \$100 in one direction or the other. Every money column is a multiple of \$100, so this is rounding, not corruption. |
| GPA range                                   | 1.5 to 4.0.                                                                                                                                               |
| Selectivity percentile range                | 16 to 102. A percentile cannot exceed 100, so a few values are out of range. They were left as they are and no test covers this yet.                      |
| Graduates with no debt                      | 11,418 (38.1%).                                                                                                                                           |
| Net ROI range                               | -\$305,200 to \$2,755,100.                                                                                                                                |

## 2.5 Limitations

The file is best read as a teaching dataset, not as a measurement of the real US education system. Several things point that way. Net cost is almost the same for every major, at \$87,433 to \$90,013 on average. Student debt is almost the same for every major, at about \$36,000 to \$40,000. Graduates are spread almost evenly across majors, at 4.5% to 6.0% each. And every money value is a multiple of \$100. Real data is rarely this uniform, so the values look simulated.

That means the strong patterns, such as STEM against arts, are properties of this file and should not be quoted as facts about US universities. Other limits to keep in mind:

- The data covers the US only and a 10 year window only. There is no graduation year, so nothing can be said about trends over time.
- Every result in this document is an average or a share. Averages hide spread, and differences between groups show association, not cause.
- Some groups are small. For example, only 73 arts graduates attended an ivy\_elite school, so that cell in the major by tier table is unreliable.

## 3. Methodology

### 3.1 Approach: load first, transform in the warehouse

The pipeline follows the ELT pattern: extract, load, then transform. The raw file is loaded into Snowflake without changing it, and all cleaning and reshaping happens afterwards, inside the warehouse, in SQL that dbt runs. This keeps the raw data intact, so any fix can be changed and re-run without going back to the source.

Three tools split the work, and a fourth runs them:

- **Snowflake** stores the data and does the computing.
- **dbt** holds all the transformations as SQL files, tests them and documents them.
- **Airflow** runs the steps in order and records whether each one passed.
- **Docker** packages Airflow and dbt so they run the same way every time.

### 3.2 Architecture

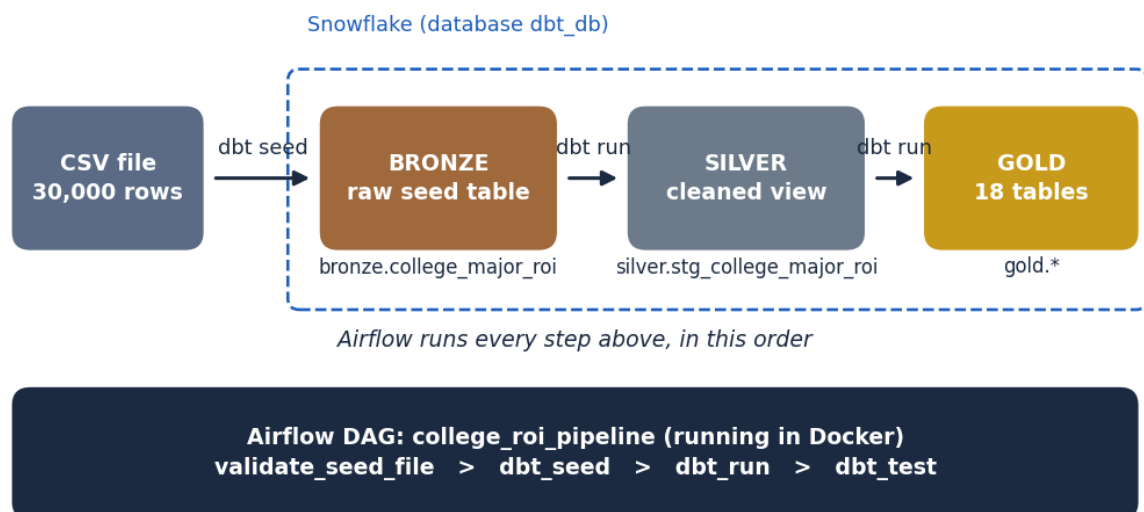


Figure 1. Data flow through the three layers, and the Airflow DAG that runs it.

### 3.3 The three layers

The data moves through three layers, often called the medallion pattern. Each layer has one job and lives in its own Snowflake schema.

| Layer  | Schema | What lives here                                                                            | Built by   | Type   |
|--------|--------|--------------------------------------------------------------------------------------------|------------|--------|
| Bronze | bronze | The CSV exactly as supplied. 30,000 rows, nothing changed.                                 | dbt seed   | Table  |
| Silver | silver | Cleaned, typed and deduplicated data with two quality flag columns: stg_college_major_roi. | dbt model  | View   |
| Gold   | gold   | 18 aggregated tables, each answering one question.                                         | dbt models | Tables |

### 3.4 Design decisions

- **Flag rows, do not delete them.** Silver keeps every row and adds `roi_calc_mismatch` and `has_negative_value` so that problems are visible and the decision to exclude anything is made later, on purpose.
- **Raw data stays untouched.** Bronze is the file as it arrived. Every correction is code that can be read and re-run.
- **One path through the layers.** Only silver reads bronze, using `source()`. Gold reads only silver, using `ref()`. This gives a single clean lineage graph.
- **Views for silver, tables for gold.** A view is cheap and always current. Gold tables are small aggregates that dashboards will read often, so they are stored.
- **Schema names controlled by a macro.** dbt normally joins the default schema and the custom schema, giving names like `dbt_schema_bronze`. A small macro override makes the layers land in plain `bronze`, `silver` and `gold`.
- **Small, cheap compute.** The warehouse is X-Small and suspends after 60 seconds, which stretches the trial credits.

### 3.5 Testing approach

Testing happened in two steps. First the data was investigated by hand with SQL: nulls, negative values and the ROI arithmetic. Then each finding that should stay true was turned into a dbt test so it is checked on every run instead of only once.

Three kinds of test are used. `not_null` checks a column has no empty values. `unique` checks no value repeats, used on `grad_id` and on the group column of each gold table. `accepted_values` checks a column only holds values from a fixed list, used on the three category columns. The suite has 49 tests: 12 on silver and 37 on gold.

Tests only report. A failing test does not change or remove any row. The fix is made upstream in the model, then the model and tests are run again.

### 3.6 Orchestration approach

One Airflow workflow, called a DAG, holds four tasks in a fixed order. Each task starts only if the one before it succeeded, so bad input stops the run early instead of producing wrong tables.

| Order | Task               | What it does                                                           |
|-------|--------------------|------------------------------------------------------------------------|
| 1     | validate_seed_file | Python check that the CSV exists in the seeds folder and is not empty. |
| 2     | dbt_seed           | Loads the CSV into <code>bronze.college_major_roi</code> .             |
| 3     | dbt_run            | Builds the silver view and the 18 gold tables.                         |
| 4     | dbt_test           | Runs all 49 tests.                                                     |

The DAG has no schedule and is started by hand. Because `dbt seed` replaces the whole table each time, editing the CSV and triggering the DAG again is enough to refresh everything, including new rows. Airflow uses LocalExecutor, which runs tasks inside the scheduler process. That suits a single machine and needs no extra worker service.

### 3.7 Tools and versions

| Tool                   | Version or setting                                                                     |
|------------------------|----------------------------------------------------------------------------------------|
| Snowflake              | Trial account: Standard edition, AWS, Asia Pacific (Sydney). \$400 credit, 30 days.    |
| Python                 | 3.13 on Windows for dbt. Python 3.12 inside the Airflow image.                         |
| dbt                    | dbt-core 1.12.5 with dbt-snowflake 1.12.1, in a virtual environment at E:\dbt\dbt-env. |
| Docker                 | Docker Desktop, Docker 28.3.3.                                                         |
| Airflow                | 2.10.3 (image apache/airflow:2.10.3 with dbt added), LocalExecutor.                    |
| Airflow metadata store | Postgres 13 container.                                                                 |

## 4. Procedure

This section is a step by step build guide. Follow it top to bottom. It is split by tool: Snowflake, then dbt, then Airflow and Docker. Every command is shown in the order it was run, followed by a short explanation of each line.

Commands run in three different places, and the label above each code block says which:

| Label             | Where to run it                                                        |
|-------------------|------------------------------------------------------------------------|
| SNOWFLAKE SQL     | A SQL worksheet in the Snowflake web app (Projects, then Worksheets).  |
| POWERSHELL        | Windows PowerShell on the local machine.                               |
| BASH IN CONTAINER | A shell inside the Airflow Docker container, opened with docker exec.  |
| FILE              | Contents of a file to create or edit, with the path in the step title. |

### 4.1 Snowflake

#### Step 1. Create the trial account

- Open the Snowflake sign up page. It shows two tabs: **AI Data Cloud** (for enterprise) and **Snowflake CoCo** (for developers). Choose **AI Data Cloud**. That is the full platform with warehouses, databases and \$400 of trial credit for 30 days. CoCo is Snowflake's AI coding assistant, not the data platform, and comes with a much smaller credit pool.
- Fill in first name, last name, email, company name, job title and country (Australia).
- Choose **Standard** edition, cloud provider **AWS** and region **Asia Pacific (Sydney)**. Enterprise is pre-selected by default. It adds features such as multi-cluster warehouses and longer Time Travel, which this project does not use, and Standard credits go further. The region is hard to change later, so pick it now.
- Tick the terms box. On the short survey that follows, tick SQL and Python. The survey only tailors the welcome screens and locks nothing in.
- Afterwards, Admin then Accounts lists the new account, and the bottom left of the app shows the trial balance (\$400 of \$400 left, trial ends in 30 days).

#### Step 2. Find the account identifier and username

dbt connects using an identifier in the form **ORGNAME-ACCOUNTNAME**. The Accounts page only shows the account name, which is not enough on its own. Run this in a worksheet:

##### SNOWFLAKE SQL

```
select current_organization_name() || '-' || current_account_name();  
select current_user();
```

- **current\_organization\_name()** returns the organisation part.
- **current\_account\_name()** returns the account part. The **||** operators join them with a hyphen, giving the value dbt needs.
- **current\_user()** returns the login name, used in the grant statements below.

#### Step 3. Create the warehouse, database and role

A dedicated warehouse, database and role keep dbt separate from the default account setup and make permissions explicit. Run this as ACCOUNTADMIN:

## SNOWFLAKE SQL

```
use role accountadmin;

create warehouse if not exists dbt_wh
  warehouse_size = 'xsmall' auto_suspend = 60 auto_resume = true initially_suspended = true;
create database if not exists dbt_db;
create schema if not exists dbt_db.dbt_schema;
create role if not exists dbt_role;

grant role dbt_role to user shaqxe;
grant usage, operate on warehouse dbt_wh to role dbt_role;
grant all on database dbt_db to role dbt_role;
grant all on schema dbt_db.dbt_schema to role dbt_role;
```

- `use role accountadmin;` switches to the top role, which is allowed to create all of these objects.
- `create warehouse ... dbt_wh` creates the compute cluster. `xsmall` is the smallest size. `auto_suspend = 60` stops it after 60 idle seconds so it stops using credits. `auto_resume = true` starts it again on the next query. `initially_suspended = true` means it does not start running until it is needed.
- `create database ... dbt_db` creates the database that holds every layer.
- `create schema ... dbt_db.dbt_schema` creates the default schema named in the dbt profile. The bronze, silver and gold schemas are created by dbt itself the first time it builds into them, so no manual step is needed for those.
- `create role ... dbt_role` creates the role dbt logs in as.
- `grant role dbt_role to user shaqxe;` lets the user take that role. Replace shaqxe with your own username.
- `grant usage, operate on warehouse ...` lets the role run queries on the warehouse and start or stop it.
- `grant all on database ... / schema ...` gives the role full rights inside the database, including creating new schemas.

## Step 4. Look at the results in Snowflake

After dbt has run (sections 4.2 and 4.3), everything can be browsed in the Snowflake web app under Catalog, then Databases, then DBT\_DB. The three schemas BRONZE, SILVER and GOLD appear there. Clicking a table and choosing Data Preview shows the rows, and the Chart tab on a result can draw a quick bar chart, which is how the tables in section 5 were first checked. The exact queries used to inspect the data are shown in the dbt section, next to the step that produced each object.

## 4.2 dbt

### Step 1. Install dbt in a virtual environment

A virtual environment is an isolated Python setup. Installing dbt inside one keeps its packages away from the rest of the machine.

#### POWERSHELL

```
cd E:\dbt
python -m venv dbt-env
.\dbt-env\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install dbt-core dbt-snowflake
dbt --version
```

- `cd E:\dbt` moves into the folder that will hold the environment and the project.

- `python -m venv dbt-env` creates the environment in a folder called dbt-env.
- `.\dbt-env\Scripts\Activate.ps1` switches this terminal to use it. The prompt then starts with (dbt-env). This must be repeated in every new terminal.
- `python -m pip install --upgrade pip` updates pip, the package installer. Optional.
- `pip install dbt-core dbt-snowflake` installs the dbt engine and the Snowflake adapter. Both are needed, because since dbt 1.8 the adapter no longer brings dbt-core with it.
- `dbt --version` confirms the install. It should show dbt-core 1.12.5 and the snowflake plugin 1.12.1.

If PowerShell refuses to run the activation script, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once and try again. Do not run `pip install dbt`. That installs a different package and breaks (see Discussion, problem 1).

## Step 2. Create the project and the connection profile

### POWERSHELL

```
cd E:\dbt
dbt init
```

`dbt init` asks a series of questions. The answers used:

| Prompt                                | Answer                                                  |
|---------------------------------------|---------------------------------------------------------|
| Enter a name for your project         | dbt_shaqxe                                              |
| Which database would you like to use? | 1 (snowflake)                                           |
| account                               | ORNAME-ACCOUNTNAME from step 2 of the Snowflake section |
| user                                  | shaqxe (the Snowflake username)                         |
| Desired authentication type           | 1 (password)                                            |
| password                              | the Snowflake password                                  |
| role                                  | dbt_role                                                |
| warehouse                             | dbt_wh                                                  |
| database                              | dbt_db                                                  |
| schema                                | dbt_schema                                              |
| threads                               | 4                                                       |

**Threads** is how many models dbt builds at the same time. Independent models can run in parallel, so 4 is faster than 1 on a project with many gold tables, and it is a normal default for an X-Small warehouse.

dbt saves the answers in a file called `profiles.yml`, kept in the user profile folder, not in the project. That keeps the password out of the project folder. It looks like this:

FILE: C:\USERS\<WINDOWS-USERNAME>\.DBT\PROFILES.YML

```
dbt_shaqxe:
  outputs:
    dev:
      account: <ORNAME>-<ACCOUNTNAME>
      database: dbt_db
      password: <your Snowflake password>
      role: dbt_role
      schema: dbt_schema
      threads: 4
      type: snowflake
      user: shaqxe
      warehouse: dbt_wh
  target: dev
```

To reopen it later: `notepad $env:USERPROFILE\.dbt\profiles.yml`. The account line is the one most likely to need fixing.

### Step 3. Test the connection

#### POWERSHELL

```
cd E:\dbt\dbt_shaqxe
dbt debug
dbt run
```

- `cd E:\dbt\dbt_shaqxe` moves into the project folder, the one that contains `dbt_project.yml`. Every dbt command must be run from here.
- `dbt debug` checks the profile and tries to log in to Snowflake. The last line should read "All checks passed!".
- `dbt run` builds the two example models that `dbt init` created. Success shows the connection, permissions and warehouse all work.

After this works, delete the folder `models\example`. It is starter content and is not part of the project.

### Step 4. Configure the layers in `dbt_project.yml`

Open `dbt_project.yml` in the project folder and make it match this:

FILE: E:\DBT\DBT\_SHAQXE\DBT\_PROJECT.YML

```
name: 'dbt_shaqxe'
version: '1.0.0'
profile: 'dbt_shaqxe'

model-paths: ["models"]
analysis-paths: ["analyses"]
test-paths: ["tests"]
seed-paths: ["seeds"]
macro-paths: ["macros"]
snapshot-paths: ["snapshots"]

clean-targets:
  - "target"
  - "dbt_packages"

models:
  dbt_shaqxe:
    silver:
      +materialized: view
      +snowflake_warehouse: dbt_wh
      +schema: silver
    gold:
      +materialized: table
      +snowflake_warehouse: dbt_wh
      +schema: gold

seeds:
  dbt_shaqxe:
    +schema: bronze
    +quote_columns: false
    docs:
      +enabled: false
```

- The top block names the project and the profile to use. The profile name must match the top line of `profiles.yml`.
- The path lines tell dbt where each kind of file lives.
- Under `models:`, the keys `silver` and `gold` must match the folder names under `models\` exactly, or dbt warns about unused configuration paths.

- The **+** in front of each setting marks it as a configuration for everything in that folder. Leaving the plus off is deprecated and produces a warning.
- **+schema: silver** and **+schema: gold** send each folder's models to their own schema. Without **+schema**, models land in the default schema, **dbt\_schema**.
- Under **seeds:**, **+schema: bronze** puts every loaded CSV in the bronze schema. **+quote\_columns: false** stops dbt wrapping column names in quotes, so Snowflake keeps them as normal uppercase names. The **docs** block with **+enabled: false** tells dbt not to load anything in **seeds/docs**, where the data dictionary sits as reference material.

## Step 5. Add the schema naming macro

By default dbt joins the profile schema and the custom schema, which turns **bronze** into **dbt\_schema\_bronze**. This macro replaces that rule.

FILE: E:\DBT\DBT\_SHAQXE\MACROS\GENERATE\_SCHEMA\_NAME.SQL

```
{% macro generate_schema_name(custom_schema_name, node) -%}
  {%- if custom_schema_name is none -%}
    {{ target.schema }}
  {%- else -%}
    {{ custom_schema_name | trim }}
  {%- endif -%}
{%- endmacro -%}
```

- The curly bracket and percent syntax is Jinja, a templating language. dbt runs it before any SQL is sent to Snowflake. The file ends in **.sql** because that is where dbt looks for macros, but it does not produce a query.
- **generate\_schema\_name** is the exact name of a macro built into dbt. Defining one with the same name in the project overrides it.
- If no custom schema is set, the macro returns the profile schema. If one is set, it returns that name on its own, with nothing added.
- The dashes inside the tags, as in **{%- if -%}**, trim blank space so the result is clean.

## Step 6. Create the folder layout and add the data

- Create these folders under the project: **models\bronze**, **models\silver**, **models\gold**, and **seeds/docs**. The **seeds** and **macros** folders already exist from **dbt init**.
- Copy **college\_major\_roi.csv** into **seeds\**. The file name becomes the table name.
- Copy **data\_dictionary.csv** into **seeds/docs\** as reference only. The full layout is in Appendix D.

## Step 7. Load the CSV into bronze

POWERSHELL

```
dbt seed --full-refresh
```

**dbt seed** reads every CSV in the seeds folder and creates a table for each. **--full-refresh** drops and recreates the table, which was needed once because the first seed had landed in the wrong schema. Expected output ends with a line reading OK loaded seed file bronze.college\_major\_roi, followed by CREATE 30000 and a time of about 72 seconds. About 70 to 80 seconds for 30,000 rows is normal, because seeds are inserted row by row. That is fine for small files. Much larger files should be loaded with Snowflake's own load tools instead.

Check the table from a Snowflake worksheet:

## SNOWFLAKE SQL

```
select * from dbt_db.bronze.college_major_roi limit 10;

select count(*) from dbt_db.bronze.college_major_roi;
-- should return 30000

select column_name, data_type
from dbt_db.information_schema.columns
where table_name = 'COLLEGE_MAJOR_ROI';
-- check the types against the data dictionary
```

## Step 8. Declare bronze as a source

FILE: E:\DBT\DBT\_SHAQXE\MODELS\BRONZE\SOURCES.YML

```
version: 2

sources:
  - name: bronze
    schema: bronze
    tables:
      - name: college_major_roi
```

This tells dbt that a raw table called `college_major_roi` already exists in the `bronze` schema and is not built by dbt. Models can then read it with `{{ source('bronze', 'college_major_roi') }}` instead of hard coding the full table name, and the lineage graph shows it as the starting point.

## Step 9. Build the silver model

FILE: E:\DBT\DBT\_SHAQXE\MODELS\SILVER\STG\_COLLEGE\_MAJOR\_ROI.SQL

```
select
  grad_id,
  trim(lower(major)) as major,
  trim(lower(major_category)) as major_category,
  trim(lower(institution_tier)) as institution_tier,
  institution_selectivity_pctile,
  gpa,
  had_internship::boolean as had_internship,
  completed_on_time::boolean as completed_on_time,
  trim(lower(region)) as region,
  net_cost_usd,
  debt_usd,
  hs_baseline_10yr_usd,
  added_earnings_10yr_usd,
  earnings_10yr_usd,
  net_roi_usd,
  roi_pct,
  positive_roi::boolean as positive_roi,
  high_roi::boolean as high_roi,
  abs((added_earnings_10yr_usd - net_cost_usd - net_roi_usd) > 100 as roi_calc_mismatch,
  (net_cost_usd < 0 or debt_usd < 0 or hs_baseline_10yr_usd < 0) as has_negative_value
from {{ source('bronze', 'college_major_roi') }}
qualify row_number() over (partition by grad_id order by grad_id) = 1
```

- `trim(lower(...))` on the four text columns removes stray spaces and forces lower case, so "STEM" and "stem" cannot become two categories. This is why the categories appear in lower case from here on.
- `::boolean` turns the 0 and 1 integers in `had_internship`, `completed_on_time`, `positive_roi` and `high_roi` into real true and false values.
- `roi_calc_mismatch` recalculates net ROI from its parts and flags any row that differs from the stored value by more than \$100. The tolerance exists because the stored value is rounded to the nearest \$100 (see Discussion, problem 6).
- `has_negative_value` flags any row where cost, debt or the high school baseline is below zero.

- `qualify row_number() over (partition by grad_id ...) = 1` keeps one row per graduate. It is a Snowflake shortcut for filtering on a window function without a subquery. It does nothing if the ids are already unique, and works as a safety net.

#### POWERSHELL

```
dbt run --select stg_college_major_roi
```

`--select` builds only the named model. Because of the folder configuration, it is created as a view in `dbt_db.silver`.

Then check the data quality flags and nulls in Snowflake:

#### SNOWFLAKE SQL: FLAG COUNTS

```
select
  count(*) as total_rows,
  sum(case when roi_calc_mismatch then 1 else 0 end) as roi_mismatches,
  sum(case when has_negative_value then 1 else 0 end) as negative_values
from dbt_db.silver.stg_college_major_roi;
```

Result: 30,000 rows, 0 ROI mismatches, 0 negative values. The first version of the flag, before the \$100 tolerance was added, returned 7,554 mismatches. To see why, this query lists the differences:

#### SNOWFLAKE SQL: INVESTIGATING THE MISMATCH

```
select
  grad_id, net_cost_usd, added_earnings_10yr_usd, net_roi_usd,
  (added_earnings_10yr_usd - net_cost_usd) as calculated_roi,
  net_roi_usd - (added_earnings_10yr_usd - net_cost_usd) as diff
from dbt_db.silver.stg_college_major_roi
where roi_calc_mismatch
limit 20;
```

Every difference was exactly +100 or -100, which showed rounding rather than bad data. Finally the null check, which returned zero for every column:

#### SNOWFLAKE SQL: NULL COUNTS

```
select
  count(*) as total_rows,
  count(*) - count(grad_id) as null_grad_id,
  count(*) - count(major) as null_major,
  count(*) - count(major_category) as null_major_category,
  count(*) - count(institution_tier) as null_institution_tier,
  count(*) - count(institution_selectivity_pctile) as null_selectivity,
  count(*) - count(gpa) as null_gpa,
  count(*) - count(region) as null_region,
  count(*) - count(net_cost_usd) as null_net_cost,
  count(*) - count(debt_usd) as null_debt,
  count(*) - count(hs_baseline_10yr_usd) as null_hs_baseline,
  count(*) - count(added_earnings_10yr_usd) as null_added_earnings,
  count(*) - count(earnings_10yr_usd) as null_earnings,
  count(*) - count(net_roi_usd) as null_net_roi,
  count(*) - count(roi_pct) as null_roi_pct
from dbt_db.silver.stg_college_major_roi;
```

`count(column)` skips nulls, so `count(*) - count(column)` gives the number of nulls in that column, for every column in one pass.

## Step 10. Add tests to silver

FILE: E:\DBT\DBT\_SHAQXE\MODELS\SILVER\SCHEMA.YML

```
version: 2

models:
  - name: stg_college_major_roi
    description: Cleaned and typed college major ROI data, deduplicated on grad_id.
    columns:
```

```

- name: grad_id
  description: Unique graduate identifier.
  tests:
    - unique
    - not_null
- name: major
  tests:
    - not_null
- name: major_category
  tests:
    - not_null
    - accepted_values:
        arguments:
          values: ['stem', 'health', 'business', 'social_science', 'education', 'humanities', 'arts']
- name: institution_tier
  tests:
    - not_null
    - accepted_values:
        arguments:
          values: ['ivy_elite', 'top_public', 'mid_tier', 'regional', 'for_profit']
- name: region
  tests:
    - not_null
    - accepted_values:
        arguments:
          values: ['northeast', 'south', 'midwest', 'west']
- name: gpa
  tests:
    - not_null
- name: net_cost_usd
  tests:
    - not_null
- name: net_roi_usd
  tests:
    - not_null

```

This file turns the manual checks into automatic ones. **unique** and **not\_null** on **grad\_id** confirm one row per graduate. **not\_null** guards the columns the gold tables rely on. **accepted\_values** locks the three category columns to the values in the data dictionary. The **arguments:** wrapper is the current dbt syntax for test inputs. Without it, dbt prints a deprecation warning.

#### POWERSHELL

```
dbt test --select stg_college_major_roi
```

Expected: 12 tests, all PASS. A failing test does not change any data. It only reports, and the fix is made in the model.

### Step 11. Build the 18 gold tables

Create one SQL file per table in **models\gold\**. Each file reads the silver model with **{{ ref('stg\_college\_major\_roi') }}**, groups the rows and calculates averages, shares or ranks. The full SQL for all 18 is in Appendix A. The tables are:

| File                    | Question it answers                                                  |
|-------------------------|----------------------------------------------------------------------|
| roi_by_major_category   | How does return differ across the 7 major categories?                |
| roi_by_institution_tier | How does return and cost differ by school type?                      |
| roi_by_debt_bucket      | How does return change as student debt grows?                        |
| roi_stem_vs_nonstem     | STEM against everything else, on return, cost, debt and internships. |
| roi_by_major            | All 19 majors ranked by average net ROI.                             |
| roi_by_internship       | Does an internship change return, within each major category?        |
| roi_by_completion       | What is the link between finishing on time, cost and return?         |
| roi_by_region           | Return, cost and debt by US region.                                  |

| File                          | Question it answers                                       |
|-------------------------------|-----------------------------------------------------------|
| roi_by_gpa_bucket             | Does a higher GPA go with a higher return?                |
| roi_by_selectivity_bucket     | Does a more selective school pay off?                     |
| roi_major_category_by_tier    | Cross tab: each category at each tier.                    |
| debt_to_earnings_ratio        | How large is debt against the earnings premium, by major? |
| major_popularity              | How many graduates take each major?                       |
| institution_tier_distribution | How are graduates spread across school types?             |
| region_roi_ranking            | Regions ranked by net ROI.                                |
| region_hs_baseline            | Regions ranked by high school baseline earnings.          |
| completion_rate_by_tier       | Share of graduates finishing on time, by school type.     |
| region_cost_comparison        | Regions ranked by average cost, with lowest and highest.  |

#### POWERSHELL

```
dbt run --select gold
```

`--select gold` builds every model in the gold folder. Each is created as a table in `dbt_db.gold` because of the folder configuration. Running plain `dbt run` builds silver and gold together, 19 models in all.

### Step 12. Add tests to gold

Create `models\gold\schema.yml` with an entry for each of the 18 models. Every table gets `not_null` on its grouping column, and `unique` where the grouping column should appear once. `roi_by_major` also tests `roi_rank`. Appendix B lists every test. In total that is 37 gold tests, which with the 12 silver tests makes 49.

#### POWERSHELL

```
dbt test --select gold
dbt test
```

- `dbt test --select gold` runs the 37 gold tests only.
- `dbt test` runs all 49 tests in the project.

### Step 13. Generate the documentation and lineage graph

#### POWERSHELL

```
cd E:\dbt\dbt_shaqxe
Remove-Item -Recurse -Force target
dbt docs generate --no-partial-parse
dbt docs serve --port 8081
```

- `Remove-Item -Recurse -Force target` deletes the cached build folder. This matters once Airflow has been running, because Docker leaves a cache with Linux style paths that Windows dbt cannot read (Discussion, problem 12). If the folder does not exist, the error can be ignored.
- `dbt docs generate --no-partial-parse` builds the documentation site from the project. The flag tells dbt to ignore any partial cache and parse everything fresh.
- `dbt docs serve --port 8081` starts a small local web server. Port 8081 is used because Airflow already holds 8080. Stop it with Ctrl+C.

Open <http://localhost:8081>. The lineage graph is behind a small graph icon in the bottom right corner. Click it, then the full screen icon. The graph shows the bronze source feeding `stg_college_major_roi`, which feeds all 18 gold tables. The seed file also appears as a separate box on the left. To hide it, type

`source:dbt_shaqxe.bronze.college_major_roi+` in the select box at the bottom of the graph and press Update Graph.

## 4.3 Airflow and Docker

Airflow runs inside Docker containers. Docker is what makes it possible to run Airflow on Windows, because Airflow itself expects Linux.

### Step 1. Start Docker Desktop and check it

Install Docker Desktop, open it from the Start menu and wait until the bottom left of its window says **Engine running**. Then, in PowerShell:

#### POWERSHELL

```
docker --version  
docker ps
```

- `docker --version` prints the version (28.3.3 here). This only proves the command line tool is installed.
- `docker ps` lists running containers. An empty table means the engine is running. If it prints an error about `dockerDesktopLinuxEngine` and a missing file, Docker Desktop is not running yet, so wait for it to finish starting.

### Step 2. Create the Airflow folder and get the starter files

#### POWERSHELL

```
mkdir E:\airflow  
cd E:\airflow  
curl -o docker-compose.yaml "https://airflow.apache.org/docs/apache-airflow/2.10.3/docker-compose.yaml"  
mkdir dags, logs, plugins, config  
"AIRFLOW_UID=50000" | Out-File -Encoding ascii .env
```

- `mkdir E:\airflow` creates a folder for Airflow, separate from the dbt project.
- `cd E:\airflow` moves into it. Every Docker command below runs from here.
- `curl -o docker-compose.yaml "..."` downloads Airflow's official docker-compose file. It is a recipe that tells Docker which containers Airflow needs and how they connect. It is replaced with the final version in step 4.
- `mkdir dags, logs, plugins, config` creates four folders that the containers share with Windows. `dags` holds the pipeline file, `logs` holds run history, and the other two are unused here.
- `"AIRFLOW_UID=50000" | Out-File ... .env` writes a one line settings file that Docker reads automatically. It sets the user id the containers run as, which avoids file permission problems.

### Step 3. Create the Dockerfile

Airflow's stock image does not contain dbt. This Dockerfile builds a custom image with dbt installed once, so it does not have to be reinstalled every time a container starts (see Discussion, problem 13).

#### FILE: E:\AIRFLOW\DOCKERFILE (NO FILE EXTENSION)

```
FROM apache/airflow:2.10.3  
RUN pip install --no-cache-dir dbt-core dbt-snowflake
```

- `FROM apache/airflow:2.10.3` starts from the official Airflow image.
- `RUN pip install --no-cache-dir dbt-core dbt-snowflake` installs dbt and the Snowflake adapter into the image. `--no-cache-dir` keeps the image smaller.

## Step 4. Replace docker-compose.yaml with the final version

Replace the whole contents of `E:\airflow\docker-compose.yaml` with the file below. It is the stock file with these changes:

- `build: .` replaces the `image:` line, so Docker builds from the Dockerfile.
- `AIRFLOW__CORE__EXECUTOR: LocalExecutor` replaces CeleryExecutor. The Redis service, the worker service and the Celery settings are removed (Discussion, problem 14).
- `AIRFLOW__CORE__LOAD_EXAMPLES: 'false'` hides Airflow's 74 built in example DAGs.
- `_PIP_ADDITIONAL_REQUIREMENTS` is empty, since dbt is now inside the image.
- Two extra volumes are added. The first mounts the dbt project into the container at `/opt/airflow/dbt_shaqxe`, so Airflow can run it. The second mounts the folder that holds `profiles.yml` at `/home/airflow/.dbt`, the place dbt looks for credentials. Replace `<windows-username>` with the real Windows user name.

FILE: E:\AIRFLOW\DOCKER-COMPOSE.YAML

```
x-airflow-common:
  &airflow-common
  build: .
  environment:
    &airflow-common-env
    AIRFLOW__CORE__EXECUTOR: LocalExecutor
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@postgres/airflow
    AIRFLOW__CORE__FERNET_KEY: ''
    AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION: 'true'
    AIRFLOW__CORE__LOAD_EXAMPLES: 'false'
    AIRFLOW__API__AUTH_BACKENDS: 'airflow.api.auth.backend.basic_auth,airflow.api.auth.backend.session'
    AIRFLOW__SCHEDULER__ENABLE_HEALTH_CHECK: 'true'
    _PIP_ADDITIONAL_REQUIREMENTS: ''
  volumes:
    - ${AIRFLOW_PROJ_DIR:-.}/dags:/opt/airflow/dags
    - ${AIRFLOW_PROJ_DIR:-.}/logs:/opt/airflow/logs
    - ${AIRFLOW_PROJ_DIR:-.}/config:/opt/airflow/config
    - ${AIRFLOW_PROJ_DIR:-.}/plugins:/opt/airflow/plugins
    - E:/dbt/dbt_shaqxe:/opt/airflow/dbt_shaqxe
    - C:/Users/<windows-username>/dbt:/home/airflow/.dbt
  user: "${AIRFLOW_UID:-50000}:0"
  depends_on:
    &airflow-common-depends-on
    postgres:
      condition: service_healthy

services:
  postgres:
    image: postgres:13
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow
    volumes:
      - postgres-db-volume:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "airflow"]
      interval: 10s
      retries: 5
      start_period: 5s
    restart: always

  airflow-webserver:
    <<: *airflow-common
    command: webserver
    ports:
      - "8080:8080"
    healthcheck:
      test: ["CMD", "curl", "--fail", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 5
```

```

    start_period: 30s
    restart: always
    depends_on:
      <<: *airflow-common-depends-on
    airflow-init:
      condition: service_completed_successfully

airflow-scheduler:
  <<: *airflow-common
  command: scheduler
  healthcheck:
    test: ["CMD", "curl", "--fail", "http://localhost:8974/health"]
    interval: 30s
    timeout: 10s
    retries: 5
    start_period: 30s
  restart: always
  depends_on:
    <<: *airflow-common-depends-on
  airflow-init:
    condition: service_completed_successfully

airflow-triggerer:
  <<: *airflow-common
  command: triggerer
  healthcheck:
    test:
      - "CMD-SHELL"
      - 'airflow jobs check --job-type TriggererJob --hostname "${HOSTNAME}"'
    interval: 30s
    timeout: 10s
    retries: 5
    start_period: 30s
  restart: always
  depends_on:
    <<: *airflow-common-depends-on
  airflow-init:
    condition: service_completed_successfully

airflow-init:
  <<: *airflow-common
  entrypoint: /bin/bash
  command:
    - -c
    - |
      mkdir -p /sources/logs /sources/dags /sources/plugins
      chown -R "${AIRFLOW_UID:-50000}:0" /sources/{logs,dags,plugins}
      exec /entrypoint airflow version
  environment:
    <<: *airflow-common-env
    _AIRFLOW_DB_MIGRATE: 'true'
    _AIRFLOW_WWW_USER_CREATE: 'true'
    _AIRFLOW_WWW_USER_USERNAME: ${_AIRFLOW_WWW_USER_USERNAME:-airflow}
    _AIRFLOW_WWW_USER_PASSWORD: ${_AIRFLOW_WWW_USER_PASSWORD:-airflow}
  user: "0:0"
  volumes:
    - ${AIRFLOW_PROJ_DIR:-.}/sources

airflow-cli:
  <<: *airflow-common
  profiles:
    - debug
  environment:
    <<: *airflow-common-env
    CONNECTION_CHECK_MAX_COUNT: "0"
  command:
    - bash
    - -c
    - airflow

volumes:
  postgres-db-volume:

```

## Step 5. Build and start Airflow

### POWERSHELL

```
cd E:\airflow
docker compose down
docker compose build
docker compose up airflow-init
docker compose up -d
docker ps
```

- `docker compose down` stops and removes any running containers. Skip it the first time. It is used whenever the compose file changes.
- `docker compose build` builds the custom image from the Dockerfile. It takes a few minutes the first time and seconds afterwards, because Docker caches the layers.
- `docker compose up airflow-init` runs the one time setup: it creates Airflow's database tables and the login user. It ends with `airflow-init-1 exited with code 0`, and code 0 means success. It is not a hang.
- `docker compose up -d` starts the real services in the background. `-d` means detached, so the terminal is returned.
- `docker ps` lists the containers. Four should be running: webserver, scheduler, triggerer and postgres. Wait about 30 seconds until they all say (healthy).

Open <http://localhost:8080> in a browser. The default login is user `airflow` and password `airflow`. Change the password if anyone else can reach the machine.

## Step 6. Check that dbt works inside the container

### POWERSHELL, THEN BASH IN CONTAINER

```
docker exec -it airflow-airflow-scheduler-1 bash
dbt --version
cd /opt/airflow/dbt_shaqxe
dbt debug
dbt run --select stg_college_major_roi
exit
```

- `docker exec -it airflow-airflow-scheduler-1 bash` opens a shell inside the scheduler container. The name comes from `docker ps`. The prompt changes to `airflow@...`
- `dbt --version` confirms dbt is in the image.
- `cd /opt/airflow/dbt_shaqxe` moves into the mounted project. Listing it should show the models, seeds and `dbt_project.yml`.
- `dbt debug` the connection line should say OK. The final line may report that git cannot be run by this user. That check is harmless here, since the project uses no git packages.
- `dbt run --select stg_college_major_roi` proves dbt can build a model in Snowflake from inside Docker.
- `exit` leaves the container. Commands such as `docker ps` only work outside it.

If `dbt run` fails with a `KeyError` naming a macros path, the cached build folder was written by Windows. Fix it inside the container with `rm -rf target/ logs/` and run it again.

## Step 7. Write the DAG

### FILE: E:\AIRFLOW\DAGS\COLLEGE\_ROI\_PIPELINE.PY

```
# Import Python's datetime, used to give the DAG a start date
from datetime import datetime
# Import DAG, the core Airflow class that represents "a pipeline"
```

```

from airflow import DAG
# Import BashOperator, a task type that runs a shell command
from airflow.operators.bash import BashOperator
# Import PythonOperator, a task type that runs a Python function
from airflow.operators.python import PythonOperator
# Import os, used here to check whether a file exists and its size
import os

# Store the dbt project's path inside the container in one variable
DBT_PROJECT_DIR = "/opt/airflow/dbt_shaqxe"

# A plain Python function that becomes the first task
def validate_seed_file():
    # Build the full path to the seed CSV inside the container
    seed_path = os.path.join(DBT_PROJECT_DIR, "seeds", "college_major_roi.csv")
    # Stop the pipeline early if the file is missing
    if not os.path.exists(seed_path):
        raise FileNotFoundError(f"Seed file missing: {seed_path}")
    # Get the file size in bytes
    size = os.path.getsize(seed_path)
    # Stop the pipeline early if the file is empty
    if size == 0:
        raise ValueError("Seed file is empty")
    # Confirmation message, visible in the task log
    print(f"Seed file found, size: {size} bytes")

# Define the DAG; everything indented below belongs to it
with DAG(
    dag_id="college_roi_pipeline",
    description="Validate, seed, transform, and test the college major ROI dataset",
    start_date=datetime(2026, 9, 1),
    schedule=None,          # no automatic schedule, manual trigger only
    catchup=False,          # do not try to run past scheduled dates
    tags=["dbt", "snowflake", "college-roi"],
) as dag:

    # Task 1: run the Python validation function
    validate = PythonOperator(
        task_id="validate_seed_file",
        python_callable=validate_seed_file,
    )

    # Task 2: load the CSV into Snowflake bronze
    seed = BashOperator(
        task_id="dbt_seed",
        bash_command=f"cd {DBT_PROJECT_DIR} && dbt seed",
    )

    # Task 3: build the silver model and the 18 gold tables
    run = BashOperator(
        task_id="dbt_run",
        bash_command=f"cd {DBT_PROJECT_DIR} && dbt run",
    )

    # Task 4: run all dbt tests
    test = BashOperator(
        task_id="dbt_test",
        bash_command=f"cd {DBT_PROJECT_DIR} && dbt test",
    )

    # Order of execution: each task starts only if the one before succeeded
    validate >> seed >> run >> test

```

How the file is put together:

- **Imports.** `DAG` represents a pipeline. `PythonOperator` runs a Python function as a task, and `BashOperator` runs a shell command as a task.
- **`validate_seed_file`.** A normal Python function. It builds the path to the CSV inside the container and raises an error if the file is missing or empty. An error makes the task fail, which stops everything after it.

- **DAG settings.** `dag_id` is the name shown in the UI. `schedule=None` means it runs only when triggered by hand. `catchup=False` stops Airflow trying to run every missed date since `start_date`. `tags` are labels for filtering.
- **The three dbt tasks.** Each one changes into the project folder and runs one dbt command. The `&&` means the second command runs only if the first worked.
- **The dependency line.** The last line, `validate >> seed >> run >> test`, sets the order. The `>>` means "then". It is what draws the connected boxes in the Airflow UI.

The validation task is deliberately simple. It checks the file exists and is not empty, but does not yet check the columns. Comparing the header against the data dictionary would be a good improvement.

## Step 8. Run the pipeline

- In the Airflow UI, wait about 30 seconds after saving the file. `college_roi_pipeline` appears in the DAG list, paused.
- Click the toggle on the left to unpause it. It turns blue.
- Click the play button on the right of the row and choose **Trigger DAG**.
- Click the DAG name to open the Grid view. Each task has a small square for each run.

| Square colour        | Meaning                                                         |
|----------------------|-----------------------------------------------------------------|
| Grey or dark outline | Queued or not started yet.                                      |
| Light green          | Running.                                                        |
| Dark green           | Succeeded.                                                      |
| Red                  | Failed. Open the task to see why.                               |
| Orange               | Upstream failed. It did not run because an earlier task failed. |

The four tasks turn dark green in order. `dbt_seed` takes over a minute and `dbt_run` takes a few minutes, so the whole run takes several minutes. When all four are dark green the pipeline has finished: the bronze table is reloaded, silver and the 18 gold tables are rebuilt and all 49 tests have passed.

If a task fails, task logs are also stored on Windows under `E:\airflow\logs`, and container logs can be read with `docker logs airflow-airflow-scheduler-1 --tail 100`.

## Step 9. Day to day use

### POWERSHELL

```
cd E:\airflow
docker compose up -d
docker compose down
```

- `docker compose up -d` starts Airflow. Docker Desktop must be running first.
- `docker compose down` stops it. The database volume is kept, so the DAG history survives.
- **To add or change data:** edit `seeds\college_major_roi.csv`, save it and trigger the DAG. The whole table is reloaded and everything downstream is rebuilt and tested.
- **To change a model:** edit the SQL file and trigger the DAG. The next run picks it up.
- **Before using dbt directly on Windows after Airflow has run,** delete the cached build folder with `Remove-Item -Recurse -Force target`.

## 5. Insights: the 18 gold tables

This section shows every table in the gold layer, exactly as built in Snowflake, followed by a chart and a plain reading of what it says. All values are averages or shares over the 30,000 graduates. Money is in US dollars over 10 years. "Net ROI" is 10 year earnings above the high school baseline, minus the net cost of the degree.

"Positive ROI" is the share of graduates whose net ROI is above zero.

Section 2.5 explains why these results describe this file and not the real US education system. Every reading below should be taken with that in mind.

### 5.0 The big picture

Before the individual tables, this summary ranks the factors by how far apart the best and worst group are on average net ROI. It is built from the tables that follow.

| Factor                      | Best group                        | Worst group           | Gap in avg net ROI  |
|-----------------------------|-----------------------------------|-----------------------|---------------------|
| Major (19 majors)           | electrical_engineering, \$600,365 | fine_arts, -\$39,821  | \$640,186           |
| Major category (7)          | stem, \$447,000                   | arts, -\$39,821       | \$486,821           |
| School type (5 tiers)       | ivy_elite, \$380,610              | for_profit, \$102,770 | \$277,840           |
| Selectivity band (4)        | top 10%, \$380,610                | bottom 50%, \$209,779 | \$170,831           |
| Finished on time (2)        | yes, \$286,991                    | no, \$223,593         | \$63,398            |
| Debt bucket (5)             | up to 25k, \$288,659              | over 100k, \$229,394  | \$59,265            |
| Internship, within category | internship                        | no internship         | \$51,094 on average |
| GPA band (4)                | 3.5 to 4.0, \$277,386             | below 2.5, \$257,427  | \$19,959            |
| Region (4)                  | midwest, \$272,086                | south, \$262,998      | \$9,088             |

The major is by far the biggest factor, then the type of school. Everything else, including region and GPA, moves the average by a comparatively small amount. The debt bucket comparison rests on a small best group of 465 graduates, so treat that one with care.

### 5.1 roi\_by\_major\_category

**Question:** How does return differ across the 7 major categories?

| major_category | Graduates | Avg net ROI | Avg ROI % | Avg net cost | Positive ROI % | High ROI % |
|----------------|-----------|-------------|-----------|--------------|----------------|------------|
| stem           | 9,904     | \$447,000   | 571.5%    | \$88,559     | 98.7%          | 45.6%      |
| health         | 1,589     | \$350,829   | 451.0%    | \$89,832     | 99.2%          | 31.0%      |
| business       | 8,361     | \$346,099   | 446.6%    | \$88,219     | 98.4%          | 29.7%      |
| social_science | 5,989     | \$31,064    | 50.7%     | \$89,024     | 66.9%          | 0.1%       |
| education      | 1,352     | \$11,548    | 25.2%     | \$88,637     | 58.1%          | 0.0%       |
| humanities     | 1,410     | -\$13,321   | -6.5%     | \$89,039     | 37.6%          | 0.0%       |
| arts           | 1,395     | -\$39,821   | -39.7%    | \$87,935     | 11.1%          | 0.0%       |

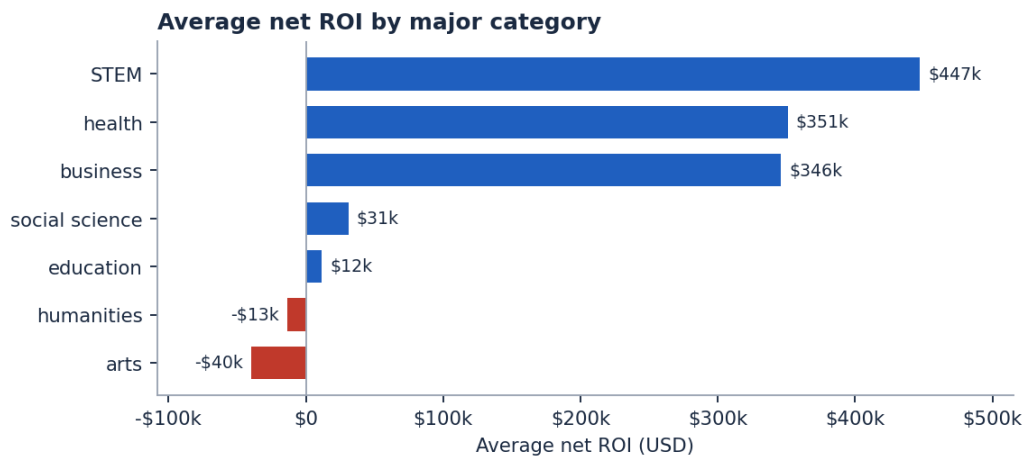


Figure 2. Average net ROI by major category. Blue is positive, red is negative.

The categories fall into three clear groups. STEM (\$447,000), health (\$350,829) and business (\$346,099) lead by a wide margin, and in each of them 98% or more of graduates finish with a positive ROI. Social science (\$31,064) and education (\$11,548) sit in the middle, with 66.9% and 58.1% positive. Humanities (-\$13,321) and arts (-\$39,821) are negative on average, and only 11.1% of arts graduates come out ahead.

Cost is almost flat across categories, from \$87,935 for arts to \$89,832 for health. That means the whole difference comes from the earnings premium, not from what the degree costs. The high ROI share shows the same split: 45.6% of STEM graduates are in the top quartile of ROI, against 31.0% for health, 29.7% for business, 0.1% for social science and none in education, humanities or arts.

## 5.2 roi\_by\_institution\_tier

**Question:** How does return and cost differ by school type?

| institution_tier | Graduates | Avg net ROI | Avg ROI % | Avg net cost | Avg selectivity pctile | Positive ROI % |
|------------------|-----------|-------------|-----------|--------------|------------------------|----------------|
| ivy_elite        | 1,752     | \$380,610   | 257.8%    | \$161,819    | 96                     | 79.7%          |
| top_public       | 5,996     | \$328,415   | 380.1%    | \$94,037     | 74                     | 85.6%          |
| mid_tier         | 10,258    | \$278,175   | 358.7%    | \$84,646     | 58                     | 85.3%          |
| regional         | 9,060     | \$244,433   | 407.0%    | \$65,322     | 42                     | 87.5%          |
| for_profit       | 2,934     | \$102,770   | 97.7%     | \$119,695    | 22                     | 63.2%          |

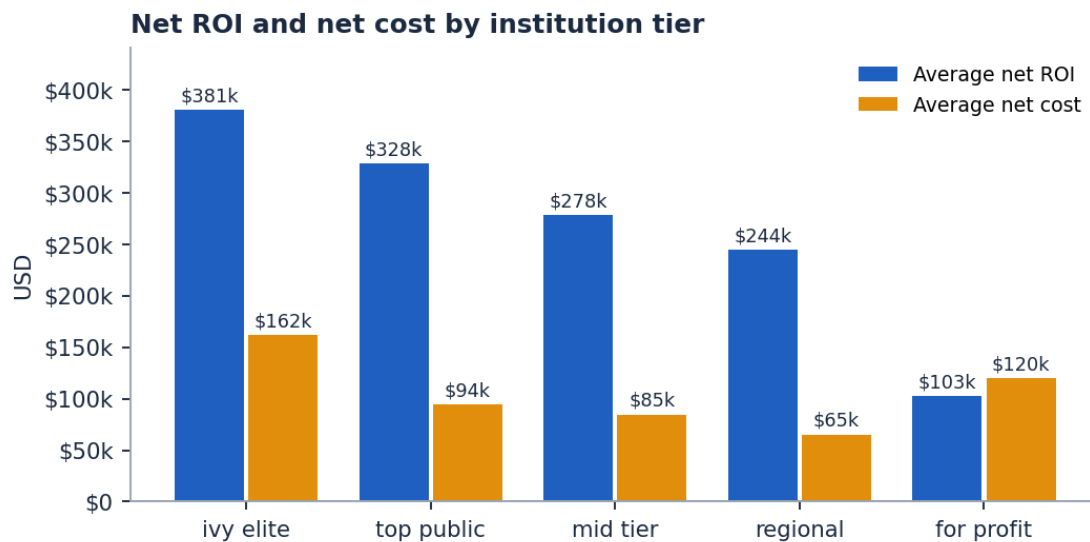


Figure 3. Net ROI and net cost by institution tier.

Average net ROI in dollars falls in exactly the order of school selectivity: `ivy_elite` (\$380,610, selectivity 96), `top_public` (\$328,415, 74), `mid_tier` (\$278,175, 58), `regional` (\$244,433, 42) and `for_profit` (\$102,770, 22). But dollars are only half the story. Ivy elite schools cost \$161,819 on average, nearly double `mid_tier` at \$84,646, so their return as a share of cost is the lowest of the four traditional tiers at 257.8%. Regional schools cost only \$65,322 and return the highest percentage, 407.0%, with the highest share of positive outcomes at 87.5%.

For-profit colleges are the weak spot. They cost \$119,695, the second highest, yet return the least: \$102,770 in dollars and 97.7% of cost. Only 63.2% of their graduates have a positive ROI.

### 5.3 roi\_by\_debt\_bucket

**Question:** How does return change as student debt grows?

| debt_bucket     | Graduates | Avg net ROI | Avg ROI % | Positive ROI % |
|-----------------|-----------|-------------|-----------|----------------|
| 0 - no debt     | 11,418    | \$264,980   | 342.7%    | 83.1%          |
| 1 - up to 25k   | 465       | \$288,659   | 594.9%    | 94.2%          |
| 2 - 25k to 50k  | 6,878     | \$284,492   | 442.6%    | 89.9%          |
| 3 - 50k to 100k | 9,519     | \$262,060   | 302.5%    | 81.3%          |
| 4 - over 100k   | 1,720     | \$229,394   | 158.2%    | 69.7%          |

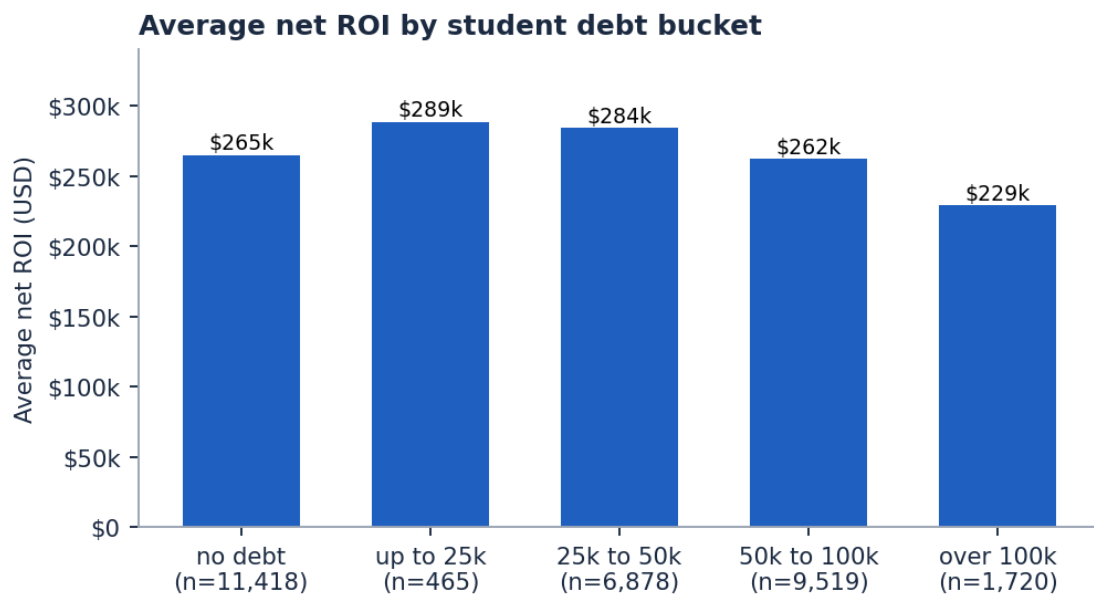


Figure 4. Average net ROI by student debt bucket, with the number of graduates in each.

The largest group, 11,418 graduates (38%), has no debt at all. Return then falls steadily as debt rises above \$50,000: \$262,060 for the \$50,000 to \$100,000 bucket and \$229,394 for debt above \$100,000, where only 69.7% of graduates have a positive ROI. Return as a percentage of cost drops from 594.9% in the smallest debt bucket to 158.2% in the largest.

Two cautions. The "up to 25k" bucket has only 465 graduates (1.6%), so its top position is a small sample result. And the table shows that higher debt goes with lower return, not that debt causes it. The file does not say why.

## 5.4 roi\_stem\_vs\_nonstem

**Question:** STEM against everything else, on return, cost, debt and internships.

| stem_flag | Graduates | Avg net ROI | Avg ROI % | Avg net cost | Avg debt | Had internship % |
|-----------|-----------|-------------|-----------|--------------|----------|------------------|
| non_stem  | 20,096    | \$178,072   | 235.1%    | \$88,652     | \$38,526 | 41.8%            |
| stem      | 9,904     | \$447,000   | 571.5%    | \$88,559     | \$38,441 | 47.3%            |

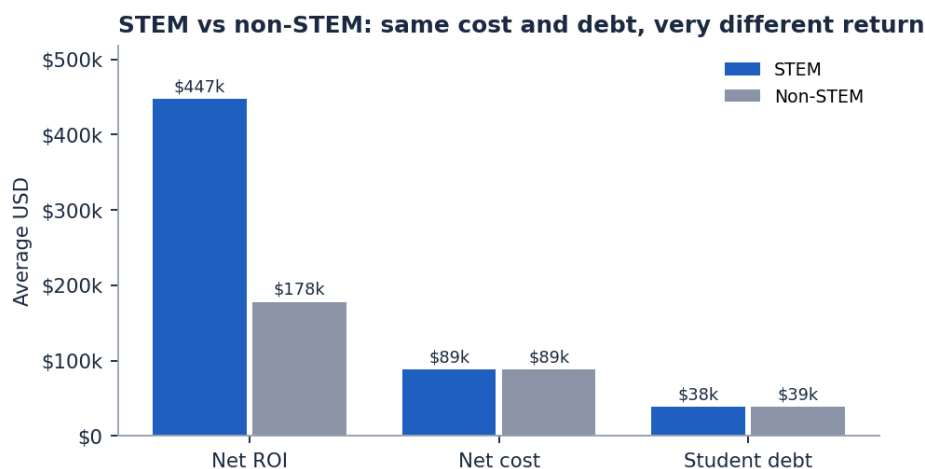


Figure 5. STEM and non-STEM graduates compared on net ROI, net cost and debt.

STEM graduates (9,904) average \$447,000 in net ROI against \$178,072 for the other 20,096, about 2.5 times as much. Yet they pay the same: \$88,559 against \$88,652 in net cost, and \$38,441 against \$38,526 in debt. The whole gap is on the earnings side. STEM graduates also do slightly more internships, 47.3% against 41.8%.

## 5.5 roi\_by\_major

**Question:** All 19 majors ranked by average net ROI.

| major                  | major_category | Graduates | Avg net ROI | Avg ROI % | Avg net cost | Avg debt | Positive ROI % | High ROI % | ROI rank |
|------------------------|----------------|-----------|-------------|-----------|--------------|----------|----------------|------------|----------|
| electrical_engineering | stem           | 1,621     | \$600,365   | 758.9%    | \$90,013     | \$39,847 | 99.9%          | 66.3%      | 1        |
| business_analytics     | business       | 1,708     | \$525,388   | 676.4%    | \$88,230     | \$39,163 | 100.0%         | 58.6%      | 2        |
| chemical_engineering   | stem           | 1,626     | \$523,860   | 667.1%    | \$88,599     | \$39,077 | 99.7%          | 57.2%      | 3        |
| computer_science       | stem           | 1,628     | \$517,458   | 658.0%    | \$88,997     | \$37,874 | 99.8%          | 56.3%      | 4        |
| computer_engineering   | stem           | 1,659     | \$495,988   | 641.4%    | \$87,920     | \$38,272 | 99.9%          | 53.7%      | 5        |
| economics              | business       | 1,580     | \$445,033   | 567.0%    | \$89,375     | \$37,990 | 99.7%          | 44.7%      | 6        |
| mechanical_engineering | stem           | 1,635     | \$427,708   | 547.9%    | \$87,815     | \$37,899 | 99.7%          | 41.5%      | 7        |
| finance                | business       | 1,676     | \$351,285   | 453.0%    | \$87,433     | \$38,455 | 99.4%          | 28.7%      | 8        |
| nursing                | health         | 1,589     | \$350,829   | 451.0%    | \$89,832     | \$39,586 | 99.2%          | 31.0%      | 9        |
| business               | business       | 1,694     | \$269,915   | 353.6%    | \$87,600     | \$36,319 | 98.2%          | 15.9%      | 10       |
| marketing              | business       | 1,703     | \$145,174   | 190.8%    | \$88,524     | \$38,433 | 94.9%          | 1.6%       | 11       |
| biology                | stem           | 1,735     | \$136,906   | 181.3%    | \$88,062     | \$37,737 | 93.4%          | 1.7%       | 12       |
| communications         | social_science | 1,788     | \$68,425    | 97.7%     | \$89,600     | \$39,678 | 82.8%          | 0.2%       | 13       |
| criminal_justice       | social_science | 1,376     | \$38,536    | 60.1%     | \$88,676     | \$39,139 | 73.9%          | 0.0%       | 14       |
| education              | education      | 1,352     | \$11,548    | 25.2%     | \$88,637     | \$38,011 | 58.1%          | 0.0%       | 15       |
| social_work            | social_science | 1,396     | \$5,908     | 18.7%     | \$89,205     | \$38,552 | 54.9%          | 0.0%       | 16       |
| psychology             | social_science | 1,429     | \$1,696     | 14.2%     | \$88,463     | \$38,433 | 51.7%          | 0.0%       | 17       |
| humanities             | humanities     | 1,410     | -\$13,321   | -6.5%     | \$89,039     | \$38,644 | 37.6%          | 0.0%       | 18       |
| fine_arts              | arts           | 1,395     | -\$39,821   | -39.7%    | \$87,935     | \$38,393 | 11.1%          | 0.0%       | 19       |

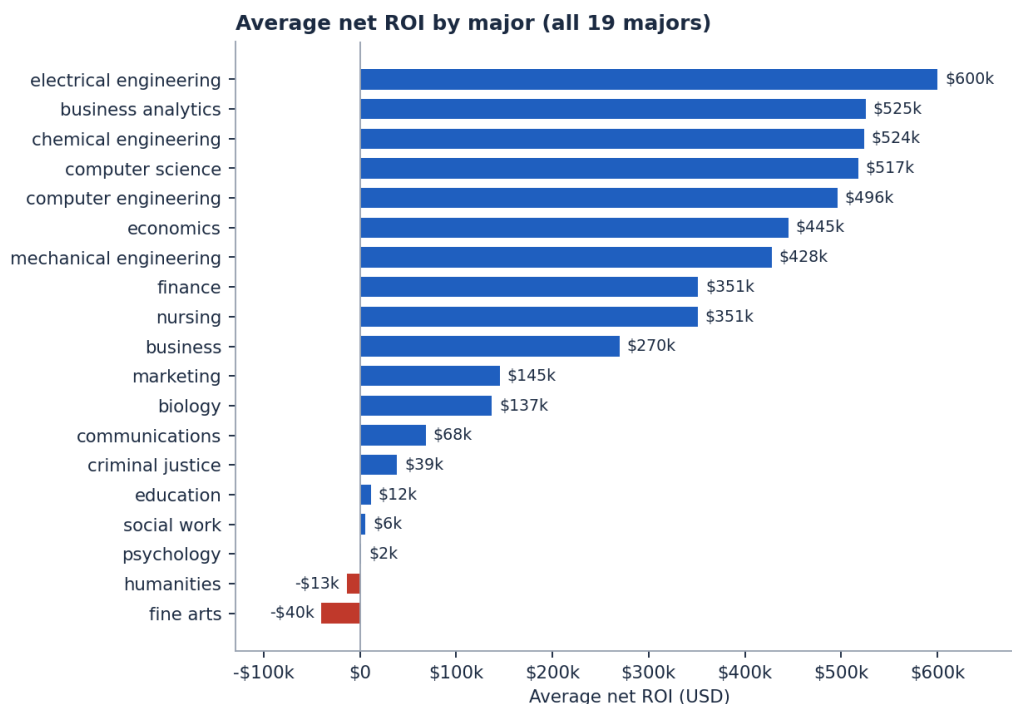


Figure 6. Average net ROI for each of the 19 majors.

Electrical engineering is first at \$600,365 (758.9% of cost), and 66.3% of its graduates land in the top quartile. Business analytics is second at \$525,388 and is the only major where every graduate has a positive ROI (100.0%). Chemical engineering, computer science and computer engineering complete the top five. At the other end, fine arts is last at -\$39,821 with only 11.1% positive, then humanities at -\$13,321. Psychology (\$1,696) and social work (\$5,908) are close to break even.

The STEM label is not uniform. Biology is classed as STEM but ranks 12th of 19 at \$136,906, behind business (\$269,915), finance (\$351,285), nursing (\$350,829) and marketing (\$145,174). Cost stays between \$87,433 and \$90,013 for every major, so the ranking is driven by earnings.

## 5.6 roi\_by\_internship

**Question:** Does an internship change return, within each major category?

| major_category | Internship | Graduates | Avg net ROI | Avg ROI % | Avg 10yr earnings |
|----------------|------------|-----------|-------------|-----------|-------------------|
| stem           | No         | 5,220     | \$406,917   | 517.5%    | \$855,244         |
| stem           | Yes        | 4,684     | \$491,669   | 631.8%    | \$939,739         |
| arts           | No         | 875       | -\$41,576   | -41.7%    | \$407,272         |
| arts           | Yes        | 520       | -\$36,868   | -36.2%    | \$411,514         |
| business       | No         | 4,408     | \$316,621   | 407.1%    | \$763,948         |
| business       | Yes        | 3,953     | \$378,971   | 490.6%    | \$827,805         |
| education      | No         | 899       | \$7,499     | 20.5%     | \$455,542         |
| education      | Yes        | 453       | \$19,583    | 34.6%     | \$467,169         |
| health         | No         | 778       | \$327,738   | 421.4%    | \$778,194         |
| health         | Yes        | 811       | \$372,980   | 479.3%    | \$824,564         |
| humanities     | No         | 885       | -\$14,602   | -8.1%     | \$433,456         |
| humanities     | Yes        | 525       | -\$11,162   | -3.9%     | \$439,737         |
| social_science | No         | 3,853     | \$26,748    | 45.1%     | \$475,662         |
| social_science | Yes        | 2,136     | \$38,849    | 60.9%     | \$487,953         |

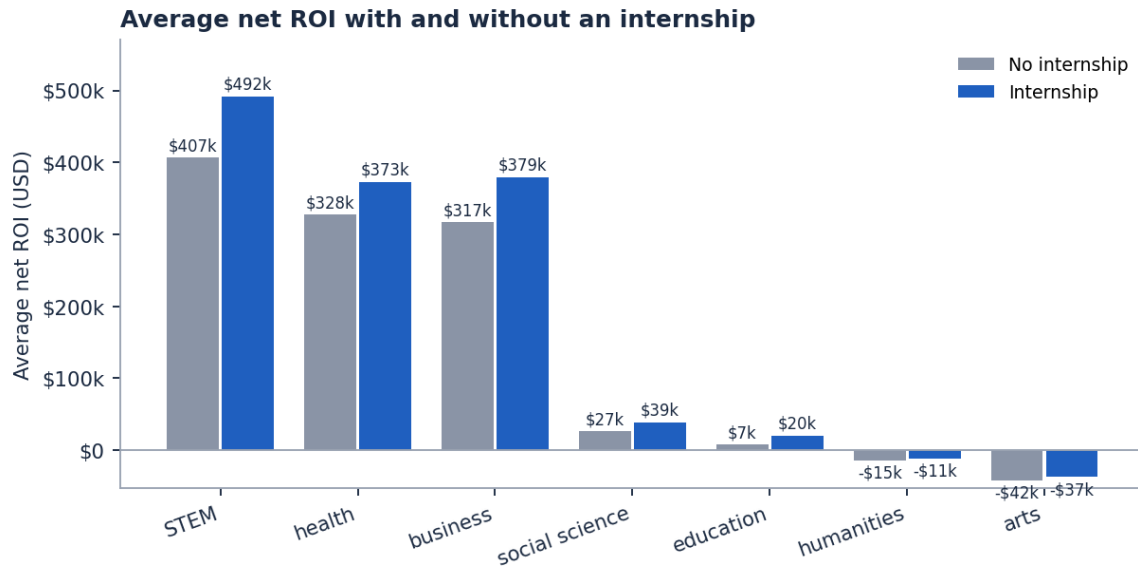


Figure 7. Average net ROI with and without an internship, by major category.

In every category the graduates with an internship have a higher average ROI, but the size of the gap varies a lot. It is \$84,752 in STEM, \$62,350 in business and \$45,242 in health. In social science and education it is about \$12,000, and in arts (\$4,709) and humanities (\$3,440) it is almost nothing.

Comparing all graduates with and without an internship gives a gap of \$92,090. That figure is inflated, because STEM and business graduates both earn more and do more internships, so the raw comparison mixes two effects. Averaging the gap within each category, weighted by group size, gives about \$51,094. That is the fairer number, and it is still an association, not proof that the internship caused the difference.

## 5.7 roi\_by\_completion

**Question:** What is the link between finishing on time, cost and return?

| Finished on time | Graduates | Avg net cost | Avg net ROI | Avg ROI % | Positive ROI % |
|------------------|-----------|--------------|-------------|-----------|----------------|
| No               | 9,529     | \$110,116    | \$223,593   | 229.6%    | 75.9%          |
| Yes              | 20,471    | \$78,616     | \$286,991   | 400.4%    | 87.1%          |

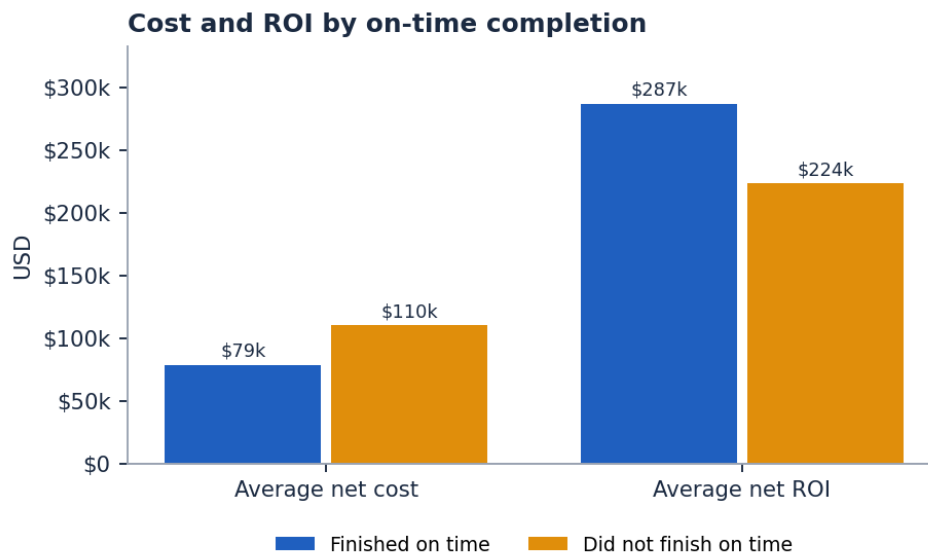


Figure 8. Net cost and net ROI for graduates who did and did not finish on time.

Most graduates, 20,471 of 30,000 (68.2%), finished on time. They paid \$78,616 on average against \$110,116 for those who did not, which is \$31,500 less. Their return is higher too: \$286,991 against \$223,593, or 400.4% of cost against 229.6%, and 87.1% have a positive ROI against 75.9%. It fits the idea that extra years cost money and delay earnings, but the file does not say which way the link runs.

## 5.8 roi\_by\_region

**Question:** Return, cost and debt by US region.

| region    | Graduates | Avg net ROI | Avg ROI % | Avg net cost | Avg debt |
|-----------|-----------|-------------|-----------|--------------|----------|
| midwest   | 6,655     | \$272,086   | 353.7%    | \$88,095     | \$38,570 |
| west      | 6,345     | \$269,229   | 351.5%    | \$88,612     | \$38,445 |
| northeast | 7,512     | \$265,082   | 342.8%    | \$88,491     | \$38,486 |
| south     | 9,488     | \$262,998   | 340.0%    | \$89,101     | \$38,492 |

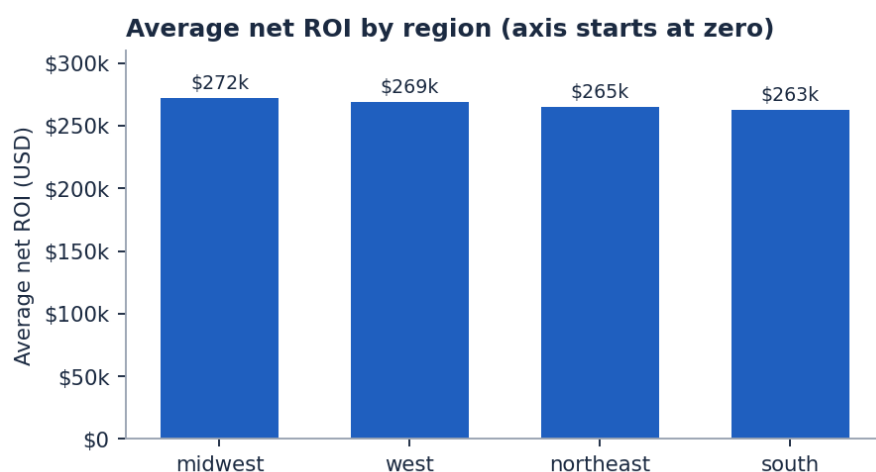


Figure 9. Average net ROI by region. The axis starts at zero so that the size of the gap is not exaggerated.

The midwest comes first at \$272,086 and the south last at \$262,998. The whole spread is \$9,088, about 3% of the average. Cost sits between \$88,095 and \$89,101 and debt between \$38,445 and \$38,570. Region is a very weak factor here compared with major or school type.

## 5.9 roi\_by\_gpa\_bucket

**Question:** Does a higher GPA go with a higher return?

| gpa_bucket     | Graduates | Avg net ROI | Avg ROI % | Positive ROI % |
|----------------|-----------|-------------|-----------|----------------|
| 1 - below 2.5  | 2,675     | \$257,427   | 334.3%    | 83.6%          |
| 2 - 2.5 to 3.0 | 9,691     | \$259,209   | 335.6%    | 82.7%          |
| 3 - 3.0 to 3.5 | 12,019    | \$270,195   | 350.7%    | 83.7%          |
| 4 - 3.5 to 4.0 | 5,615     | \$277,386   | 360.4%    | 84.4%          |

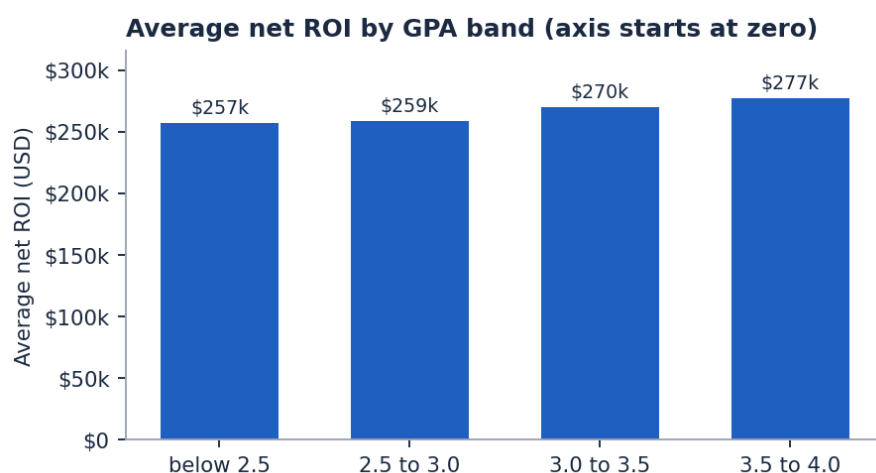


Figure 10. Average net ROI by GPA band. The axis starts at zero.

Yes, but only slightly. Average net ROI rises from \$257,427 for GPAs below 2.5 to \$277,386 for 3.5 to 4.0. The gap is \$19,959, about 8%, and the share of positive outcomes hardly changes, staying between 82.7% and 84.4%. For comparison, the gap between the best and worst major is \$640,186. The largest band is 3.0 to 3.5, with 12,019 graduates (40%).

## 5.10 roi\_by\_selectivity\_bucket

**Question:** Does a more selective school pay off?

| selectivity_bucket | Graduates | Avg net cost | Avg net ROI | Avg ROI % |
|--------------------|-----------|--------------|-------------|-----------|
| 1 - top 10%        | 1,752     | \$161,819    | \$380,610   | 257.8%    |
| 2 - top 25%        | 2,783     | \$93,840     | \$318,789   | 369.6%    |
| 3 - top 50%        | 13,471    | \$86,926     | \$292,147   | 366.0%    |
| 4 - bottom 50%     | 11,994    | \$78,623     | \$209,779   | 331.3%    |

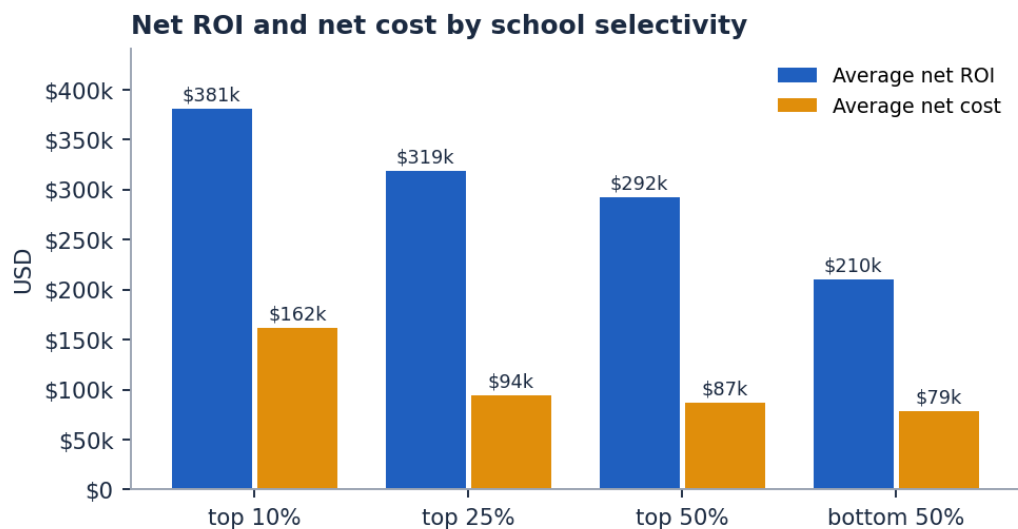


Figure 11. Net ROI and net cost by school selectivity band.

In dollars, yes: net ROI goes from \$209,779 for the bottom half of schools to \$380,610 for the top 10%. But the cost rises much faster at the top. Moving from the top 25% band to the top 10% band adds \$61,821 of ROI and \$67,979 of cost, and the return as a share of cost drops from 369.6% to 257.8%. The best value per dollar is the top 25% band.

One detail worth knowing: the top 10% band has 1,752 graduates and the same averages as the ivy\_elite row in table 5.2, because those are the same graduates. This table partly repeats that one.

## 5.11 roi\_major\_category\_by\_tier

**Question:** Cross tab: how does each category do at each school type?

| major_category | institution_tier | Graduates | Avg net ROI | Avg ROI % |
|----------------|------------------|-----------|-------------|-----------|
| stem           | for_profit       | 994       | \$214,658   | 197.1%    |
| stem           | ivy_elite        | 590       | \$644,915   | 433.2%    |
| stem           | mid_tier         | 3,420     | \$457,961   | 584.6%    |
| stem           | regional         | 2,940     | \$403,585   | 670.7%    |
| stem           | top_public       | 1,960     | \$551,250   | 631.5%    |
| arts           | for_profit       | 130       | -\$88,264   | -71.7%    |
| arts           | ivy_elite        | 73        | -\$90,840   | -53.6%    |
| arts           | mid_tier         | 497       | -\$35,719   | -38.7%    |
| arts           | regional         | 408       | -\$22,890   | -31.1%    |
| arts           | top_public       | 287       | -\$36,076   | -35.3%    |
| business       | for_profit       | 754       | \$155,450   | 143.6%    |

| major_category | institution_tier | Graduates | Avg net ROI | Avg ROI % |
|----------------|------------------|-----------|-------------|-----------|
| business       | ivy_elite        | 494       | \$499,056   | 333.7%    |
| business       | mid_tier         | 2,841     | \$360,366   | 464.6%    |
| business       | regional         | 2,560     | \$311,734   | 515.8%    |
| business       | top_public       | 1,712     | \$413,641   | 479.3%    |
| education      | for_profit       | 133       | -\$57,211   | -43.6%    |
| education      | ivy_elite        | 73        | -\$203      | 6.6%      |
| education      | mid_tier         | 454       | \$21,476    | 34.4%     |
| education      | regional         | 426       | \$19,846    | 38.2%     |
| education      | top_public       | 266       | \$18,917    | 28.2%     |
| health         | for_profit       | 182       | \$150,696   | 143.9%    |
| health         | ivy_elite        | 100       | \$461,053   | 317.5%    |
| health         | mid_tier         | 529       | \$370,730   | 478.0%    |
| health         | regional         | 471       | \$335,813   | 543.9%    |
| health         | top_public       | 307       | \$422,316   | 487.3%    |
| humanities     | for_profit       | 136       | -\$71,813   | -58.0%    |
| humanities     | ivy_elite        | 80        | -\$49,019   | -25.3%    |
| humanities     | mid_tier         | 503       | -\$6,887    | -2.5%     |
| humanities     | regional         | 419       | -\$316      | 5.1%      |
| humanities     | top_public       | 272       | -\$5,508    | -0.5%     |
| social_science | for_profit       | 605       | -\$45,669   | -34.2%    |
| social_science | ivy_elite        | 342       | \$12,449    | 14.3%     |
| social_science | mid_tier         | 2,014     | \$39,148    | 55.8%     |
| social_science | regional         | 1,836     | \$39,668    | 72.6%     |
| social_science | top_public       | 1,192     | \$48,439    | 62.1%     |

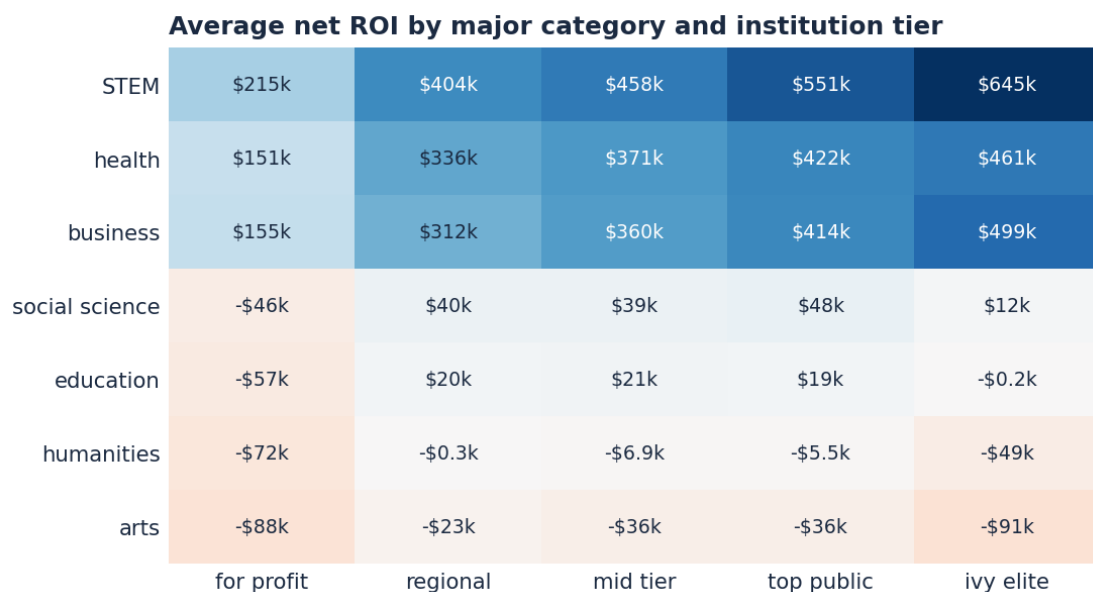


Figure 12. Average net ROI for every combination of major category and school type.

The heatmap shows that school type matters most for the high earning categories. STEM ranges from \$214,658 at for-profit schools to \$644,915 at ivy elite schools. Business goes from \$155,450 to \$499,056 and health from \$150,696 to \$461,053. For STEM, regional schools give the best percentage return, 670.7%, even though their dollar figure (\$403,585) is lower, because they cost so much less.

The low earning categories are different. Arts is negative at every school type and worst at ivy elite (-\$90,840) and for-profit (-\$88,264) schools, so paying more for a prestigious school does not help an arts degree in this data. Education, humanities and social science are all negative at for-profit schools (-\$57,211, -\$71,813 and -\$45,669). Some cells are small, for example only 73 arts graduates at ivy elite schools, so read the extremes with care.

## 5.12 debt\_to\_earnings\_ratio

**Question:** How large is student debt against the earnings premium, by major?

| major                  | major_category | Avg debt | Avg added earnings<br>10yr | Debt to earnings |
|------------------------|----------------|----------|----------------------------|------------------|
| electrical_engineering | stem           | \$39,847 | \$690,376                  | 0.071            |
| computer_science       | stem           | \$37,874 | \$606,455                  | 0.076            |
| business_analytics     | business       | \$39,163 | \$613,618                  | 0.078            |
| chemical_engineering   | stem           | \$39,077 | \$612,462                  | 0.079            |
| computer_engineering   | stem           | \$38,272 | \$583,906                  | 0.081            |
| economics              | business       | \$37,990 | \$534,408                  | 0.087            |
| mechanical_engineering | stem           | \$37,899 | \$515,523                  | 0.091            |
| finance                | business       | \$38,455 | \$438,719                  | 0.107            |
| nursing                | health         | \$39,586 | \$440,662                  | 0.111            |
| business               | business       | \$36,319 | \$357,516                  | 0.125            |
| marketing              | business       | \$38,433 | \$233,699                  | 0.199            |
| biology                | stem           | \$37,737 | \$224,968                  | 0.200            |
| communications         | social_science | \$39,678 | \$158,024                  | 0.307            |
| criminal_justice       | social_science | \$39,139 | \$127,212                  | 0.370            |
| education              | education      | \$38,011 | \$100,184                  | 0.466            |
| social_work            | social_science | \$38,552 | \$95,114                   | 0.489            |
| psychology             | social_science | \$38,433 | \$90,158                   | 0.515            |
| humanities             | humanities     | \$38,644 | \$75,717                   | 0.624            |
| fine_arts              | arts           | \$38,393 | \$48,115                   | 0.966            |

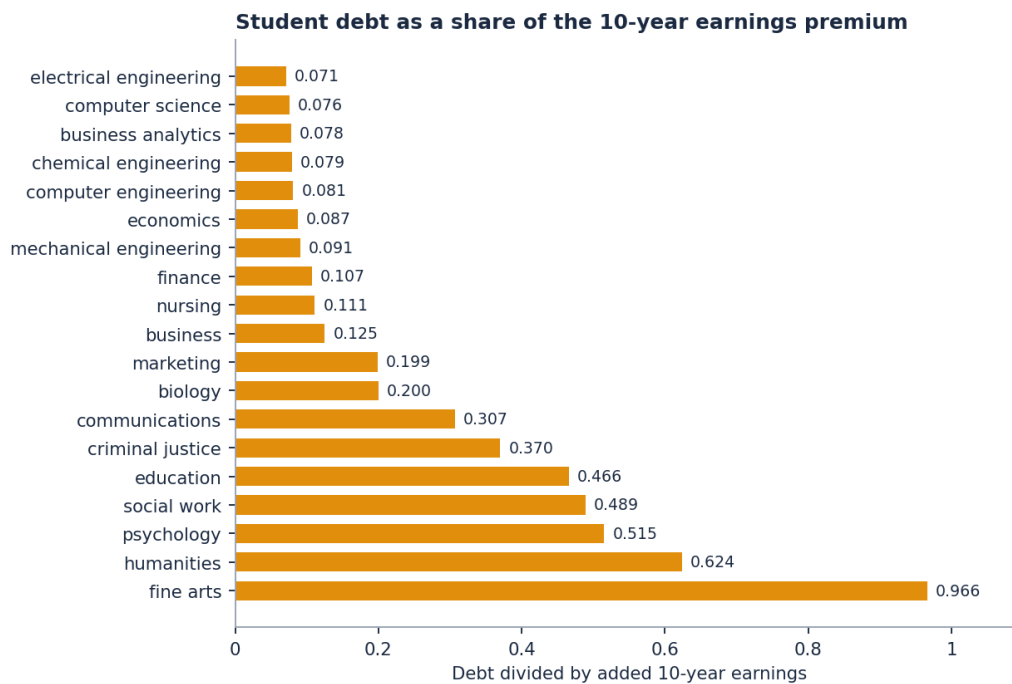


Figure 13. Average debt divided by average added 10-year earnings, by major. Lower is better.

Debt is nearly the same for every major, between \$36,319 and \$39,847, but the earnings premium is not, so the burden differs enormously. For electrical engineering, debt is 0.071 of the added earnings, about 7%. For fine arts it is 0.966: debt is almost as large as the entire 10 year earnings premium (\$38,393 of debt against \$48,115 of added earnings). Seven majors are under 0.1, and humanities (0.624), psychology (0.515) and social work (0.489) are the next worst after fine arts. The ratio is about 13.6 times higher for fine arts than for electrical engineering.

## 5.13 major\_popularity

**Question:** How many graduates take each major?

| major                  | major_category | Graduates | Share of all graduates % |
|------------------------|----------------|-----------|--------------------------|
| communications         | social_science | 1,788     | 5.96%                    |
| biology                | stem           | 1,735     | 5.78%                    |
| business_analytics     | business       | 1,708     | 5.69%                    |
| marketing              | business       | 1,703     | 5.68%                    |
| business               | business       | 1,694     | 5.65%                    |
| finance                | business       | 1,676     | 5.59%                    |
| computer_engineering   | stem           | 1,659     | 5.53%                    |
| mechanical_engineering | stem           | 1,635     | 5.45%                    |
| computer_science       | stem           | 1,628     | 5.43%                    |
| chemical_engineering   | stem           | 1,626     | 5.42%                    |
| electrical_engineering | stem           | 1,621     | 5.40%                    |
| nursing                | health         | 1,589     | 5.30%                    |
| economics              | business       | 1,580     | 5.27%                    |
| psychology             | social_science | 1,429     | 4.76%                    |
| humanities             | humanities     | 1,410     | 4.70%                    |

| major            | major_category | Graduates | Share of all graduates % |
|------------------|----------------|-----------|--------------------------|
| social_work      | social_science | 1,396     | 4.65%                    |
| fine_arts        | arts           | 1,395     | 4.65%                    |
| criminal_justice | social_science | 1,376     | 4.59%                    |
| education        | education      | 1,352     | 4.51%                    |

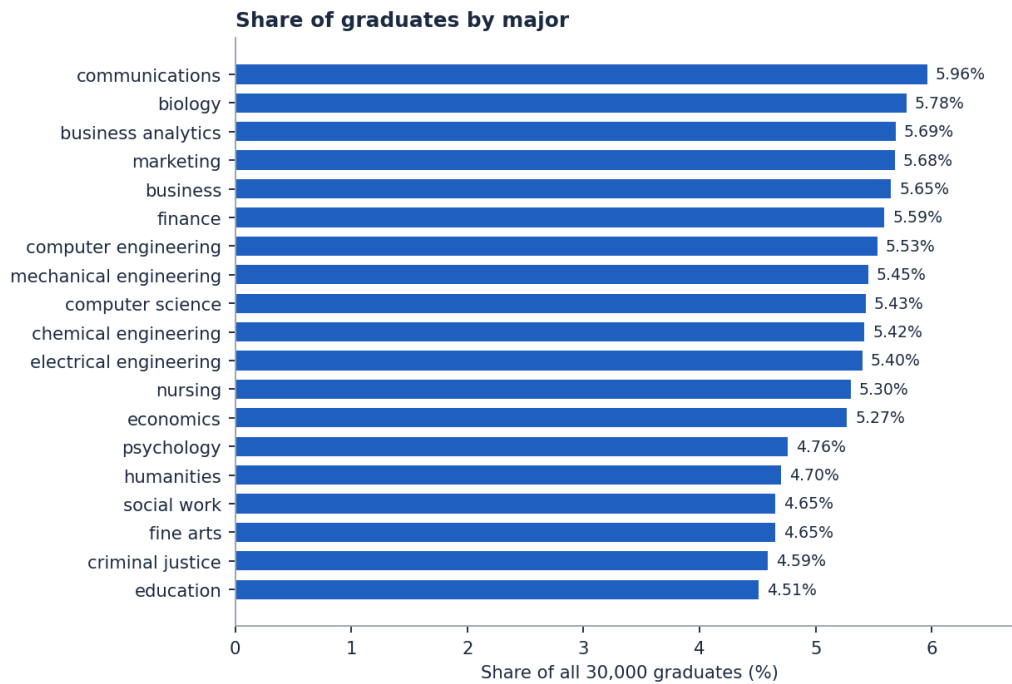


Figure 14. Share of all graduates by major.

Communications is the most common major with 1,788 graduates (5.96%) and education the least common with 1,352 (4.51%). All 19 majors sit within 1.45 percentage points of each other. Real enrolment is far more uneven, so this is another sign that the file is simulated and should not be used as a picture of real major choice.

## 5.14 institution\_tier\_distribution

**Question:** How are graduates spread across school types?

| institution_tier | Graduates | Share of all graduates % |
|------------------|-----------|--------------------------|
| mid_tier         | 10,258    | 34.19%                   |
| regional         | 9,060     | 30.20%                   |
| top_public       | 5,996     | 19.99%                   |
| for_profit       | 2,934     | 9.78%                    |
| ivy_elite        | 1,752     | 5.84%                    |

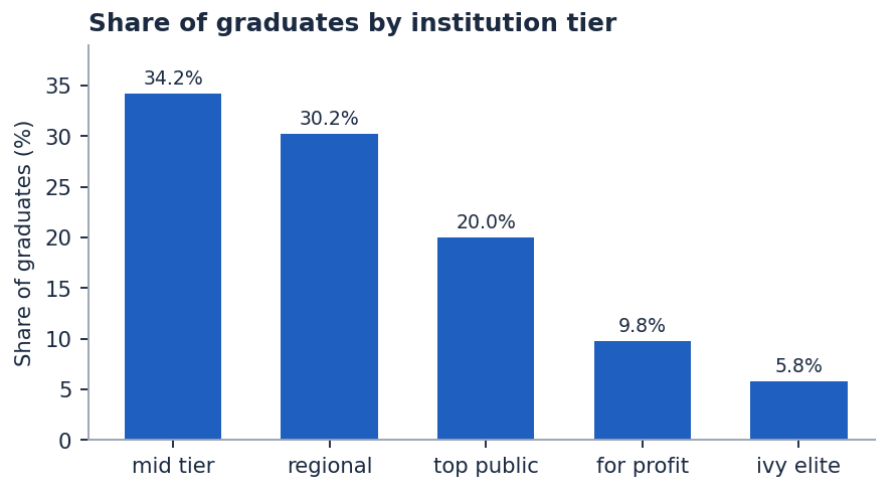


Figure 15. Share of graduates by institution tier.

Mid tier (34.19%) and regional (30.20%) schools together hold nearly two thirds of the graduates. Top public schools have 19.99%, for-profit 9.78% and ivy elite 5.84%. The ivy elite average in the earlier tables therefore rests on 1,752 graduates, much fewer than the 10,258 behind the mid tier average.

## 5.15 region\_roi\_ranking

**Question:** Regions ranked by average net ROI.

| region    | Graduates | Avg net ROI | Avg ROI % | ROI rank |
|-----------|-----------|-------------|-----------|----------|
| midwest   | 6,655     | \$272,086   | 353.7%    | 1        |
| west      | 6,345     | \$269,229   | 351.5%    | 2        |
| northeast | 7,512     | \$265,082   | 342.8%    | 3        |
| south     | 9,488     | \$262,998   | 340.0%    | 4        |

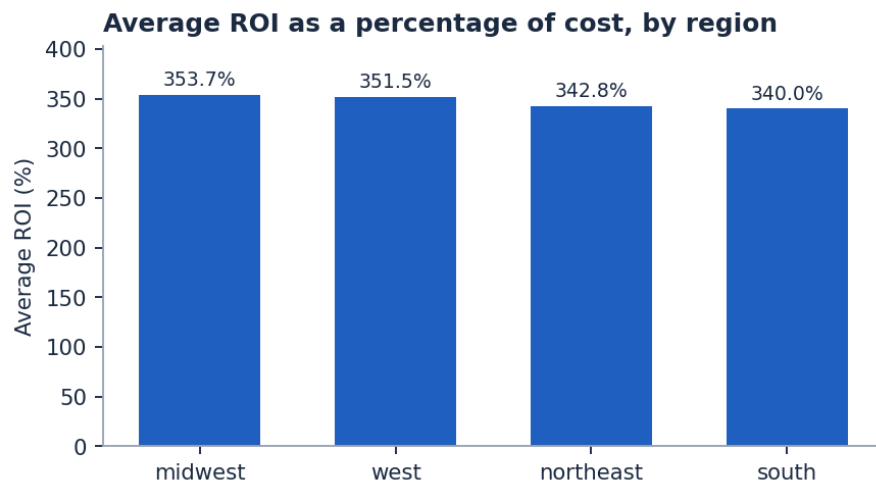


Figure 16. Average ROI as a percentage of cost, by region.

The midwest ranks first on both dollars (\$272,086) and percentage (353.7%), followed by the west, northeast and south (340.0%). The south has the most graduates (9,488) and the lowest return. In percentage terms the whole range is only 13.7 points. The ranking is real in the numbers but small, and it should not be read as a strong regional effect.

## 5.16 region\_hs\_baseline

**Question:** Regions ranked by what high school graduates would have earned.

| region    | Graduates | Avg HS baseline 10yr | Rank |
|-----------|-----------|----------------------|------|
| midwest   | 6,655     | \$360,749            | 1    |
| south     | 9,488     | \$359,777            | 2    |
| northeast | 7,512     | \$359,713            | 3    |
| west      | 6,345     | \$359,384            | 4    |

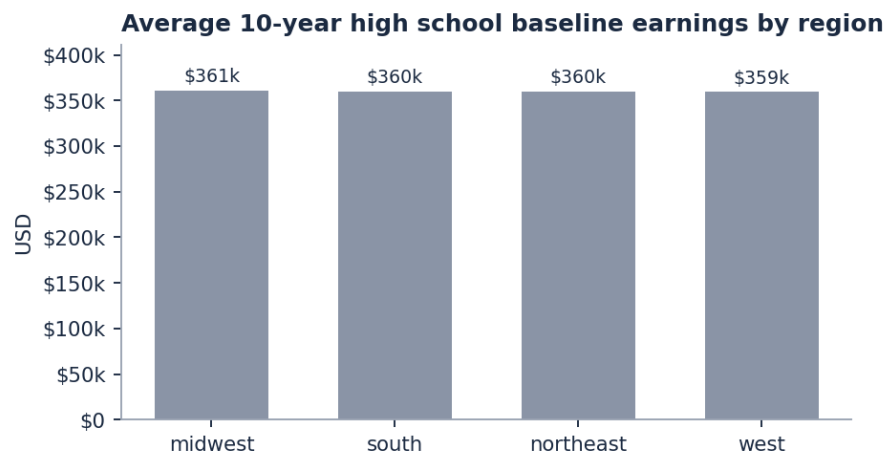


Figure 17. Average 10-year high school baseline earnings by region. The axis starts at zero.

The midwest has the highest baseline at \$360,749 and the west the lowest at \$359,384. The gap is \$1,365, about 0.4%. In this data a person who stops at high school earns about the same over 10 years wherever they live, so the small regional differences in ROI do not come from different baselines.

## 5.17 completion\_rate\_by\_tier

**Question:** What share of graduates finish on time, by school type?

| institution_tier | Graduates | On time (count) | On time % |
|------------------|-----------|-----------------|-----------|
| ivy_elite        | 1,752     | 1,231           | 70.3%     |
| regional         | 9,060     | 6,203           | 68.5%     |
| mid_tier         | 10,258    | 7,013           | 68.4%     |
| top_public       | 5,996     | 4,059           | 67.7%     |
| for_profit       | 2,934     | 1,965           | 67.0%     |

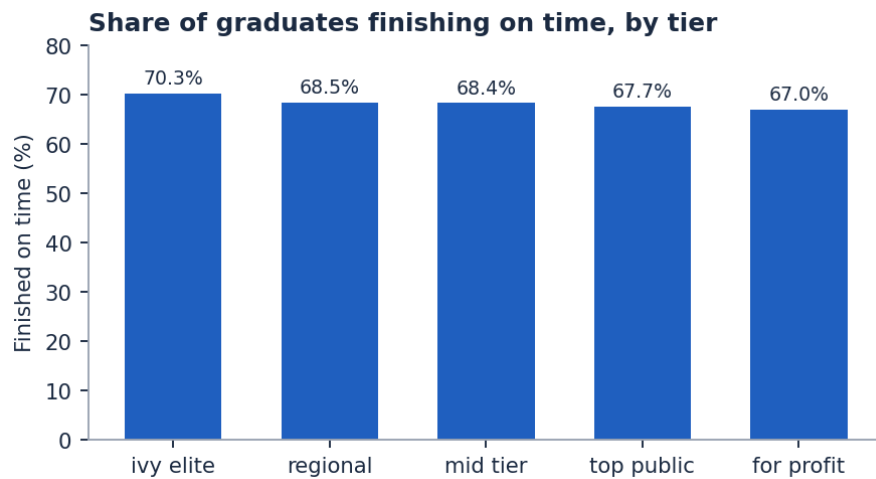


Figure 18. Share of graduates finishing on time, by institution tier.

Ivy elite schools have the highest on-time rate at 70.3% (1,231 of 1,752) and for-profit schools the lowest at 67.0% (1,965 of 2,934). The whole range is 3.3 percentage points, so school type barely changes the chance of finishing on time. That also means the on-time cost difference in table 5.7 is not explained by school type.

## 5.18 region\_cost\_comparison

**Question:** Regions ranked by average cost of study, with the lowest and highest cost in each.

| region    | Graduates | Avg net cost | Min net cost | Max net cost | Cost rank |
|-----------|-----------|--------------|--------------|--------------|-----------|
| south     | 9,488     | \$89,101     | \$29,200     | \$322,200    | 1         |
| west      | 6,345     | \$88,612     | \$32,800     | \$291,400    | 2         |
| northeast | 7,512     | \$88,491     | \$29,000     | \$334,600    | 3         |
| midwest   | 6,655     | \$88,095     | \$28,500     | \$352,900    | 4         |

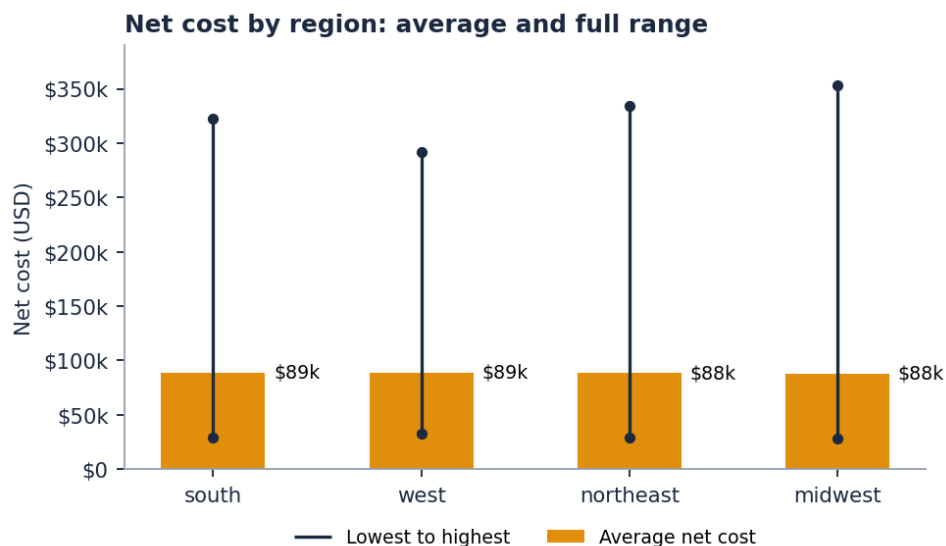


Figure 19. Average net cost by region, with the full range from lowest to highest.

The south is the most expensive on average at \$89,101 and the midwest the cheapest at \$88,095, a difference of \$1,006. The range inside each region is far bigger than the difference between regions: net cost starts between \$28,500 and \$32,800 and tops out between \$291,400 (west) and \$352,900 (midwest). Where a graduate studied says very little about what the degree cost. The type of school says much more, as table 5.2 shows.

## 6. Discussion: problems and how they were solved

---

Most of the time on this project went into things that broke, not into the analysis. This section records every problem in the order it appeared, with the symptom, the cause and the fix. Anyone rebuilding the project is likely to meet several of them. The problems fall into three groups: setting up dbt and Snowflake (1 to 8), setting up Docker and Airflow (9 to 15), and one about credentials (16).

### Setting up dbt and Snowflake

#### Problem 1. The wrong dbt package

**What happened.** After installing, `dbt init` crashed with an `ImportError: cannot import name ArtifactMixin from dbt.artifacts.schemas.base`.

**Cause.** The command used was `pip install dbt`. The package called dbt is not the same thing as dbt-core with an adapter. It installed a different set of files (dbt 2.0.6) whose parts did not match, so dbt failed as soon as it started.

**How it was fixed.** The package was removed with `pip uninstall dbt -y`. A fresh virtual environment was created (`python -m venv dbt-env`) and the right packages installed with `pip install dbt-core dbt-snowflake`. Afterwards `dbt --version` reported dbt-core 1.12.5 and the Snowflake plugin 1.12.1. Both packages are needed, because the adapter no longer pulls in dbt-core by itself.

#### Problem 2. Snowflake connection returned a 404

**What happened.** `dbt debug` failed with "290404 (08001): 404 Not Found" and a URL beginning with the bare account name.

**Cause.** Only the account name (shown on the Accounts page) had been entered. Snowflake needs the organisation name in front of it, joined with a hyphen.

**How it was fixed.** The full identifier was read from Snowflake with `select current_organization_name() || '-' || current_account_name();`, and the `account:` line in `profiles.yml` was changed to that value. `dbt debug` then passed.

#### Problem 3. Working in the wrong folder

**What happened.** `cd E:\dbt\dbt_shaqxe\dbt_shaqxe` failed with "path does not exist".

**Cause.** The path used was one level too deep. The real project folder is `E:\dbt\dbt_shaqxe`.

**How it was fixed.** File Explorer showed the true location, with `dbt_project.yml` directly inside it. A PowerShell search would also find it: `Get-ChildItem -Recurse -Filter dbt_project.yml`. The rule is that dbt commands must be run from the folder that contains `dbt_project.yml`.

#### Problem 4. Configuration warnings, and a model in the wrong schema

**What happened.** Three separate things showed up in `dbt_project.yml`. First, a deprecation warning said the `+` prefix was missing on `snowflake_warehouse`. Second, dbt warned about unused configuration paths for `staging` and `marts`. Third, after the silver model was built, the log showed it had been created as `dbt_schema.stg_college_major_roi` instead of in a silver schema.

**Cause.** The starter template used the folder names `staging` and `marts`, but the project uses `silver` and `gold`, and the keys must match the folder names exactly. Settings need a `+` in front of them. The silver folder had no

`+schema` setting, so its models fell back to the profile schema. The first run after the edit also still showed the old keys, because dbt was reusing a cached parse.

**How it was fixed.** The keys were renamed to `silver` and `gold`, the `+` prefix added, and `+schema: silver` and `+schema: gold` added. `dbt clean` cleared the stale cache. The model was rebuilt with `dbt run --select stg_college_major_roi --full-refresh` and appeared as `silver.stg_college_major_roi`. A remaining warning about unused paths for silver and gold simply means the folders were empty and disappears once they hold models.

### Problem 5. Schemas named with a prefix

**What happened.** `dbt seed` worked, but the table landed in `dbt_schema_bronze` instead of `bronze`.

**Cause.** dbt's default rule joins the profile schema and the custom schema with an underscore.

**How it was fixed.** The `generate_schema_name` macro was added (section 4.2, step 5) to return the custom name on its own. Then `dbt seed --full-refresh` recreated the table as `bronze.college_major_roi`. The earlier copy in `dbt_schema_bronze` is not removed automatically and can be dropped (Appendix E).

### Problem 6. A quarter of the rows failed the ROI check

**What happened.** The `roi_calc_mismatch` flag was true for 7,554 of 30,000 rows, which is 25%. That is too many to be random errors.

**Cause.** The first version of the flag treated any difference as a mismatch. The listing query showed that every difference was exactly +100 or -100. Every money value in the file is a multiple of \$100, so the stored `net_roi_usd` had simply been rounded to the nearest hundred, while its two components had not.

**How it was fixed.** The flag was changed to `abs((added_earnings_10yr_usd - net_cost_usd) - net_roi_usd) > 100`. The count dropped to 0. The lesson: investigate a large number of bad rows before deleting anything, because the rule may be wrong rather than the data.

### Problem 7. Deprecation warning on accepted\_values

**What happened.** Every `dbt test` run printed "MissingArgumentsPropertyInGenericTestDeprecation".

**Cause.** Newer dbt versions want the inputs of a generic test wrapped in an `arguments:` key.

**How it was fixed.** In `schema.yml`, the `values:` list under each `accepted_values` test was moved one level down under `arguments:`. The tests behave the same, and the warning went away.

### Problem 8. dbt seed was slow

**What happened.** Loading the 30,000 row CSV took 72 to 81 seconds.

**Cause.** Seeds are meant for small reference files. dbt inserts the rows through the connector rather than bulk loading them.

**How it was fixed.** Nothing needed fixing. For much larger files, the Snowflake web upload or a bulk `COPY INTO` load is the better route.

## Setting up Docker and Airflow

### Problem 9. Docker was installed but not running

**What happened.** `docker --version` worked, but `docker ps` failed with "open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file specified".

**Cause.** The version command only needs the command line tool. Running containers needs the Docker engine, which starts with Docker Desktop.

**How it was fixed.** Docker Desktop was opened from the Start menu. Once its window showed "Engine running", `docker ps` returned an empty table.

### Problem 10. 74 example DAGs in the Airflow UI

**What happened.** The first look at the Airflow UI showed 74 DAGs, none of them the project's.

**Cause.** Airflow ships with built in example DAGs and loads them by default.

**How it was fixed.** In `docker-compose.yml`, `AIRFLOW__CORE__LOAD_EXAMPLES` was changed from `'true'` to `'false'`, and the containers were restarted with `docker compose down` and `docker compose up -d`. The list was then empty and ready for the project DAG.

### Problem 11. Getting dbt to work inside the container

**What happened.** To run dbt from Airflow, the dbt project and the credentials file had to be visible inside the container, and dbt had to be installed there. The first test, `dbt debug` inside the scheduler, said `bash: dbt: command not found`. Later, once dbt was found, the connection test said OK, but the last check failed with "Error from git --help: User does not have permissions for this command: git".

**Cause.** Airflow's stock image does not contain dbt. At that stage dbt was being installed at every container start through `_PIP_ADDITIONAL_REQUIREMENTS`, which takes a couple of minutes, and the command had been run too early. The git message comes from a check dbt runs for packages fetched from git. The container user is not allowed to run git in the mounted folder, and the project uses no git packages.

**How it was fixed.** Two volumes were added to the compose file: the project folder, mounted at `/opt/airflow/dbt_shaqxe`, and the folder that holds `profiles.yml`, mounted at `/home/airflow/.dbt`. After waiting for the install, `dbt --version` worked. The git message was ignored, since `dbt run` worked regardless. The install method itself was replaced later (problem 13).

### Problem 12. A cache written by Windows broke dbt in the container, and the reverse

**What happened.** Inside the container, `dbt run` failed with a long traceback ending in `KeyError: 'dbt_shaqxe://macros/generate_schema_name.sql'`. Later the same kind of `KeyError`, with backslashes in the path, appeared on Windows when running `dbt docs generate` after Airflow had been running.

**Cause.** dbt stores its parsed view of the project in the `target` folder. That folder sits inside the project, which is shared between Windows and the Linux container. Windows and Linux write file paths differently, so each side failed to read the cache the other had written.

**How it was fixed.** In the container: `rm -rf target/ logs/`, then `dbt run` worked (19 models, 1 seed, 49 tests, 1 source). On Windows: `Remove-Item -Recurse -Force target`, then `dbt docs generate --no-partial-parse`. To avoid it for good, set `DBT_TARGET_PATH` to a folder inside the container in the compose environment, so the container never writes into the shared folder. That was noted but not applied.

### Problem 13. Tasks stuck in the queue while the worker kept restarting

**What happened.** The first DAG run got stuck. `validate_seed_file`, a check that takes a fraction of a second, stayed queued for minutes, and the tasks after it stayed grey. The Event Log showed entries such as "stuck in queued". `docker ps` showed the worker as "Up 3 seconds (health: starting)", again and again. A second, separate symptom: `docker compose build` finished instantly with no output.

**Cause.** The worker was restarting. At that time it re-checked and reinstalled dbt on every start (the container log printed "Installing additional requirements" each time, with Airflow's own warning that this is fragile), which made start up slow. The instant build happened because the `build: .` line in the compose file was commented out and the plain `image:` line was still active, so Docker had nothing to build. It was found with `Select-String -Path E:\airflow\docker-compose.yaml -Pattern "build:image:"`.

**How it was fixed.** A Dockerfile was written that installs dbt once into a custom image, the `image:` line was swapped for `build: .`, and `docker compose build` produced the image. Afterwards `dbt --version` worked in the container at once, with no install step. The restarts did not stop, though, because there was a second cause (problem 14).

#### Problem 14. The Celery worker crashed on start, so LocalExecutor replaced it

**What happened.** Tasks were still stuck. `docker logs airflow-airflow-worker-1 --tail 60` ended with `AttributeError: 'NoneType' object has no attribute 'split'`, raised in Celery's node name code (`celery/utils/nodenames.py`) while the worker was starting up. The container then restarted and the cycle repeated.

**Cause.** The Celery worker was failing to start while working out its own name. Adding `hostname: airflow-worker` to the worker service (it was present at line 154 of the compose file) did not stop it. The exact root cause was not established.

**How it was fixed.** Rather than keep patching, the executor was changed. With LocalExecutor, tasks run inside the scheduler process, so there is no separate worker or Redis to fail. `AIRFLOW__CORE__EXECUTOR` was set to `LocalExecutor` and the Redis service, the worker service and the Celery settings were removed. That is the compose file in section 4.3, step 4. After `docker compose down`, `docker compose build` and `docker compose up -d`, only four containers ran (webserver, scheduler, triggerer, postgres). `validate_seed_file` and `dbt_seed` went green within minutes and the full run completed. LocalExecutor is the sensible choice on a single machine anyway, because Celery is designed for spreading work over several machines.

#### Problem 15. Finding the real error

**What happened.** A red task gave no reason. The task page offered Details, Graph, Gantt, Code and Event Log, with no obvious Logs tab. Typing `docker ps` inside a container returned "Cannot connect to the Docker daemon".

**Cause.** The Docker command line is not installed inside the containers, so it cannot list containers from within one. And the task logs were missing because the task had never started running.

**How it was fixed.** The useful sources were, in order: `docker ps` from Windows to see whether a container was restarting; `docker logs <container> --tail 60` to read why; and the mounted folder `E:\airflow\logs`, searched with `Get-ChildItem -Recurse "E:\airflow\logs" -Filter "*.log"`. That search found scheduler logs but no task log. The scheduler log for the DAG file showed it was read successfully again and again, which ruled out a mistake in the DAG code. No task log plus a healthy DAG file meant the task was dying before it began, which pointed to the worker.

## Credentials

#### Problem 16. Passwords in plain sight

**What happened.** A screenshot of `profiles.yml` showed the Snowflake password in plain text.

**Cause.** `profiles.yml` stores the password unencrypted, which is normal for dbt but easy to expose by accident. The Airflow login is also the well known default.

**How it was fixed.** The file is kept outside the project folder, so it is not copied with the project or committed. Any password that has been pasted or shown on screen should be changed, in Snowflake and then in `profiles.yml`. The Airflow default login (`airflow` and `airflow`) should be changed if anyone else can reach the machine. Key pair authentication is a stronger option for dbt and is one of the suggestions in the next section.

## 7. Conclusion

---

### 7.1 What the pipeline gives you

The result is a small but complete data pipeline, and the value is in how it is put together, not in the size of the data. A raw file goes in at one end and 18 tested, documented analysis tables come out at the other, and one click in Airflow runs the whole thing in the right order.

- **It is repeatable.** Editing the CSV and triggering the DAG rebuilds everything. Nothing depends on remembering manual steps.
- **It fails early and loudly.** If the file is missing, the run stops at the first task. If the data breaks a rule, the test task fails and the run shows red, instead of wrong numbers quietly reaching a report.
- **The raw data is never changed.** Bronze keeps the file as it arrived, so a cleaning rule can be corrected and re-run at any time.
- **It is easy to extend.** A new question is one new SQL file in the gold folder. A new data file is one new CSV in seeds and one line in sources.yml. The tests and lineage graph pick them up.
- **It is documented by construction.** The dbt docs site shows every model, column, test and the way data flows between them.
- **It is cheap to run.** An X-Small warehouse that suspends after 60 seconds keeps the trial credit going a long way.

The same shape, raw data landed first, cleaned in stages, tested and run by an orchestrator, is how analytics teams handle much larger data. Only the size and the number of sources change.

### 7.2 What the dataset tells you, and what it does not

Within this file, the major dominates everything: STEM, health and business graduates average \$346,099 to \$447,000 in net ROI, while arts and humanities graduates average below zero, at almost the same cost. School type comes next, with for-profit colleges returning the least and regional colleges giving the best return per dollar spent. Debt looks roughly the same for every major, so its weight falls hardest on majors with a small earnings premium, and it reaches 0.966 of the premium for fine arts. Internships go with a higher return, mainly in STEM and business. School type barely changes the chance of finishing on time, and region and GPA move the average return only slightly.

What it cannot tell you is how the real US education system behaves. The values look simulated, the same numbers repeat across groups, and everything reported is an average or a share. It suits a portfolio project well, because it is clean and has clear structure, but the findings should be described as findings about this dataset.

### 7.3 State of the project

| Area                  | State                                                                                                    |
|-----------------------|----------------------------------------------------------------------------------------------------------|
| Pipeline              | Done. All four Airflow tasks pass. 19 models built, 49 tests pass.                                       |
| Documentation         | Done. dbt docs and lineage graph work. This document.                                                    |
| Dashboard             | Not built. The gold tables are ready for Power BI or Tableau.                                            |
| Scheduling and alerts | Not set up. The DAG runs only when triggered, and failures show in the UI only.                          |
| Failure demonstration | Not run. A deliberately bad row (for example a region called mars) would show the test task catching it. |

| Area            | State                                                                                                                                      |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Version control | Not used. The project has no git repository, so the only copy is on the local disk.                                                        |
| Snowflake trial | Runs for 30 days from sign up. After that the account needs a plan, or the tables must be rebuilt elsewhere. Rebuilding takes one DAG run. |

## 7.4 Suggested next steps

- Add a range test for the selectivity percentile (0 to 100), which the data currently breaks, and decide whether to clip or exclude those rows.
- Extend `validate_seed_file` to compare the CSV header with the data dictionary, so a wrong file is stopped before it is loaded.
- Run the failure demonstration: break one row on purpose, trigger the DAG, and record the failed test.
- Set `DBT_TARGET_PATH` in the compose file to stop the cache clash between Windows and Docker.
- Build a dashboard on the gold tables, starting with major, school type and the debt to earnings ratio.
- Add a schedule and a failure notification to the DAG.
- Move dbt to key pair authentication and put the project under version control.

## Appendix A. Gold model SQL

---

All 18 files live in `models\gold\`. Each reads the silver model with `ref()`. Patterns used: `group by` with averages, `case when` to make buckets, and window functions for shares (`sum(...) over ()`) and ranks (`rank() over (...)`).

### roi\_by\_major\_category.sql

```
select
  major_category,
  count(*) as num_graduates,
  round(avg(net_roi_usd), 0) as avg_net_roi_usd,
  round(avg(roi_pct), 1) as avg_roi_pct,
  round(avg(net_cost_usd), 0) as avg_net_cost_usd,
  round(100.0 * sum(case when positive_roi then 1 else 0 end) / count(*), 1) as pct_positive_roi,
  round(100.0 * sum(case when high_roi then 1 else 0 end) / count(*), 1) as pct_high_roi
from {{ ref('stg_college_major_roi') }}
group by major_category
order by avg_net_roi_usd desc
```

### roi\_by\_institution\_tier.sql

```
select
  institution_tier,
  count(*) as num_graduates,
  round(avg(net_roi_usd), 0) as avg_net_roi_usd,
  round(avg(roi_pct), 1) as avg_roi_pct,
  round(avg(net_cost_usd), 0) as avg_net_cost_usd,
  round(avg(institution_selectivity_pctile), 0) as avg_selectivity_pctile,
  round(100.0 * sum(case when positive_roi then 1 else 0 end) / count(*), 1) as pct_positive_roi
from {{ ref('stg_college_major_roi') }}
group by institution_tier
order by avg_net_roi_usd desc
```

### roi\_by\_debt\_bucket.sql

```
select
  case
    when debt_usd = 0 then '0 - no debt'
    when debt_usd <= 25000 then '1 - up to 25k'
    when debt_usd <= 50000 then '2 - 25k to 50k'
    when debt_usd <= 100000 then '3 - 50k to 100k'
    else '4 - over 100k'
  end as debt_bucket,
  count(*) as num_graduates,
  round(avg(net_roi_usd), 0) as avg_net_roi_usd,
  round(avg(roi_pct), 1) as avg_roi_pct,
  round(100.0 * sum(case when positive_roi then 1 else 0 end) / count(*), 1) as pct_positive_roi
from {{ ref('stg_college_major_roi') }}
group by debt_bucket
order by debt_bucket
```

### roi\_stem\_vs\_nonstem.sql

```
select
  case when major_category = 'stem' then 'stem' else 'non_stem' end as stem_flag,
  count(*) as num_graduates,
  round(avg(net_roi_usd), 0) as avg_net_roi_usd,
  round(avg(roi_pct), 1) as avg_roi_pct,
  round(avg(net_cost_usd), 0) as avg_net_cost_usd,
  round(avg(debt_usd), 0) as avg_debt_usd,
  round(100.0 * sum(case when had_internship then 1 else 0 end) / count(*), 1) as pct_had_internship
from {{ ref('stg_college_major_roi') }}
group by stem_flag
```

## roi\_by\_major.sql

```
select
  major,
  major_category,
  count(*) as num_graduates,
  round(avg(net_roi_usd), 0) as avg_net_roi_usd,
  round(avg(roi_pct), 1) as avg_roi_pct,
  round(avg(net_cost_usd), 0) as avg_net_cost_usd,
  round(avg(debt_usd), 0) as avg_debt_usd,
  round(100.0 * sum(case when positive_roi then 1 else 0 end) / count(*), 1) as pct_positive_roi,
  round(100.0 * sum(case when high_roi then 1 else 0 end) / count(*), 1) as pct_high_roi,
  rank() over (order by avg(net_roi_usd) desc) as roi_rank
from {{ ref('stg_college_major_roi') }}
group by major, major_category
order by avg_net_roi_usd desc
```

## roi\_by\_internship.sql

```
select
  major_category,
  had_internship,
  count(*) as num_graduates,
  round(avg(net_roi_usd), 0) as avg_net_roi_usd,
  round(avg(roi_pct), 1) as avg_roi_pct,
  round(avg(earnings_10yr_usd), 0) as avg_earnings_10yr_usd
from {{ ref('stg_college_major_roi') }}
group by major_category, had_internship
order by major_category, had_internship desc
```

## roi\_by\_completion.sql

```
select
  completed_on_time,
  count(*) as num_graduates,
  round(avg(net_cost_usd), 0) as avg_net_cost_usd,
  round(avg(net_roi_usd), 0) as avg_net_roi_usd,
  round(avg(roi_pct), 1) as avg_roi_pct,
  round(100.0 * sum(case when positive_roi then 1 else 0 end) / count(*), 1) as pct_positive_roi
from {{ ref('stg_college_major_roi') }}
group by completed_on_time
```

## roi\_by\_region.sql

```
select
  region,
  count(*) as num_graduates,
  round(avg(net_roi_usd), 0) as avg_net_roi_usd,
  round(avg(roi_pct), 1) as avg_roi_pct,
  round(avg(net_cost_usd), 0) as avg_net_cost_usd,
  round(avg(debt_usd), 0) as avg_debt_usd
from {{ ref('stg_college_major_roi') }}
group by region
order by avg_net_roi_usd desc
```

## roi\_by\_gpa\_bucket.sql

```
select
  case
    when gpa < 2.5 then '1 - below 2.5'
    when gpa < 3.0 then '2 - 2.5 to 3.0'
    when gpa < 3.5 then '3 - 3.0 to 3.5'
    else '4 - 3.5 to 4.0'
  end as gpa_bucket,
  count(*) as num_graduates,
  round(avg(net_roi_usd), 0) as avg_net_roi_usd,
  round(avg(roi_pct), 1) as avg_roi_pct,
  round(100.0 * sum(case when positive_roi then 1 else 0 end) / count(*), 1) as pct_positive_roi
from {{ ref('stg_college_major_roi') }}
group by gpa_bucket
order by gpa_bucket
```

### roi\_by\_selectivity\_bucket.sql

```
select
  case
    when institution_selectivity_pctile >= 90 then '1 - top 10%'
    when institution_selectivity_pctile >= 75 then '2 - top 25%'
    when institution_selectivity_pctile >= 50 then '3 - top 50%'
    else '4 - bottom 50%'
  end as selectivity_bucket,
  count(*) as num_graduates,
  round(avg(net_cost_usd), 0) as avg_net_cost_usd,
  round(avg(net_roi_usd), 0) as avg_net_roi_usd,
  round(avg(roi_pct), 1) as avg_roi_pct
from {{ ref('stg_college_major_roi') }}
group by selectivity_bucket
order by selectivity_bucket
```

### roi\_major\_category\_by\_tier.sql

```
select
  major_category,
  institution_tier,
  count(*) as num_graduates,
  round(avg(net_roi_usd), 0) as avg_net_roi_usd,
  round(avg(roi_pct), 1) as avg_roi_pct
from {{ ref('stg_college_major_roi') }}
group by major_category, institution_tier
order by major_category, avg_net_roi_usd desc
```

### debt\_to\_earnings\_ratio.sql

```
select
  major,
  major_category,
  round(avg(debt_usd), 0) as avg_debt_usd,
  round(avg(added_earnings_10yr_usd), 0) as avg_added_earnings_10yr_usd,
  round(avg(case when added_earnings_10yr_usd > 0
    then debt_usd::float / added_earnings_10yr_usd else null end), 3) as avg_debt_to_earnings_ratio
from {{ ref('stg_college_major_roi') }}
group by major, major_category
order by avg_debt_to_earnings_ratio asc
```

### major\_popularity.sql

```
select
  major,
  major_category,
  count(*) as num_graduates,
  round(100.0 * count(*) / sum(count(*) over (), 2) as pct_of_all_graduates
from {{ ref('stg_college_major_roi') }}
group by major, major_category
order by num_graduates desc
```

### institution\_tier\_distribution.sql

```
select
  institution_tier,
  count(*) as num_graduates,
  round(100.0 * count(*) / sum(count(*) over (), 2) as pct_of_all_graduates
from {{ ref('stg_college_major_roi') }}
group by institution_tier
order by num_graduates desc
```

### region\_roi\_ranking.sql

```
select
    region,
    count(*) as num_graduates,
    round(avg(net_roi_usd), 0) as avg_net_roi_usd,
    round(avg(roi_pct), 1) as avg_roi_pct,
    rank() over (order by avg(net_roi_usd) desc) as roi_rank
from {{ ref('stg_college_major_roi') }}
group by region
order by roi_rank
```

### region\_hs\_baseline.sql

```
select
    region,
    count(*) as num_graduates,
    round(avg(hs_baseline_10yr_usd), 0) as avg_hs_baseline_10yr_usd,
    rank() over (order by avg(hs_baseline_10yr_usd) desc) as hs_baseline_rank
from {{ ref('stg_college_major_roi') }}
group by region
order by hs_baseline_rank
```

### completion\_rate\_by\_tier.sql

```
select
    institution_tier,
    count(*) as num_graduates,
    sum(case when completed_on_time then 1 else 0 end) as num_completed_on_time,
    round(100.0 * sum(case when completed_on_time then 1 else 0 end) / count(*), 1) as
pct_completed_on_time
from {{ ref('stg_college_major_roi') }}
group by institution_tier
order by pct_completed_on_time desc
```

### region\_cost\_comparison.sql

```
select
    region,
    count(*) as num_graduates,
    round(avg(net_cost_usd), 0) as avg_net_cost_usd,
    round(min(net_cost_usd), 0) as min_net_cost_usd,
    round(max(net_cost_usd), 0) as max_net_cost_usd,
    rank() over (order by avg(net_cost_usd) desc) as cost_rank
from {{ ref('stg_college_major_roi') }}
group by region
order by cost_rank
```

## Appendix B. Gold model tests

Every gold model has a test on its grouping column. This is the content of `models\gold\schema.yml` in table form. There are 37 tests here and 12 on silver, which makes 49.

| Model                         | Tests                                                |
|-------------------------------|------------------------------------------------------|
| roi_by_major_category         | major_category: unique, not_null                     |
| roi_by_institution_tier       | institution_tier: unique, not_null                   |
| roi_by_debt_bucket            | debt_bucket: unique, not_null                        |
| roi_stem_vs_nonstem           | stem_flag: unique, not_null                          |
| roi_by_major                  | major: unique, not_null; roi_rank: not_null          |
| roi_by_internship             | major_category: not_null; had_internship: not_null   |
| roi_by_completion             | completed_on_time: unique, not_null                  |
| roi_by_region                 | region: unique, not_null                             |
| roi_by_gpa_bucket             | gpa_bucket: unique, not_null                         |
| roi_by_selectivity_bucket     | selectivity_bucket: unique, not_null                 |
| roi_major_category_by_tier    | major_category: not_null; institution_tier: not_null |
| debt_to_earnings_ratio        | major: unique, not_null                              |
| major_popularity              | major: unique, not_null                              |
| institution_tier_distribution | institution_tier: unique, not_null                   |
| region_roi_ranking            | region: unique, not_null                             |
| region_hs_baseline            | region: unique, not_null                             |
| completion_rate_by_tier       | institution_tier: unique, not_null                   |
| region_cost_comparison        | region: unique, not_null                             |

Example of the file format for the first two models:

```
version: 2

models:
  - name: roi_by_major_category
    description: Average ROI, cost, and outcomes grouped by major category.
    columns:
      - name: major_category
        tests:
          - unique
          - not_null
  - name: roi_by_institution_tier
    description: Average ROI, cost, and selectivity grouped by institution tier.
    columns:
      - name: institution_tier
        tests:
          - unique
          - not_null
```

## Appendix C. Quick reference

| Task                             | Command                                            | Where                                 |
|----------------------------------|----------------------------------------------------|---------------------------------------|
| Activate the dbt environment     | .\dbt-env\Scripts\Activate.ps1                     | PowerShell, from E:\dbt               |
| Test the Snowflake connection    | dbt debug                                          | PowerShell, in the project folder     |
| Load the CSV                     | dbt seed --full-refresh                            | PowerShell                            |
| Build silver only                | dbt run --select stg_college_major_roi             | PowerShell                            |
| Build gold only                  | dbt run --select gold                              | PowerShell                            |
| Build everything                 | dbt run                                            | PowerShell                            |
| Run all 49 tests                 | dbt test                                           | PowerShell                            |
| Clear the cache                  | Remove-Item -Recurse -Force target                 | PowerShell                            |
| Build the docs                   | dbt docs generate --no-partial-parse               | PowerShell                            |
| View the docs                    | dbt docs serve --port 8081                         | PowerShell, then localhost:8081       |
| Start Airflow                    | docker compose up -d                               | PowerShell, in E:\airflow             |
| Stop Airflow                     | docker compose down                                | PowerShell, in E:\airflow             |
| Rebuild the image                | docker compose build                               | PowerShell, in E:\airflow             |
| List containers                  | docker ps                                          | PowerShell                            |
| Read container logs              | docker logs airflow-airflow-scheduler-1 --tail 100 | PowerShell                            |
| Open a shell in a container      | docker exec -it airflow-airflow-scheduler-1 bash   | PowerShell                            |
| Clear the cache in the container | rm -rf target/ logs/                               | Container, in /opt/airflow/dbt_shaqxe |
| Airflow UI                       | http://localhost:8080 (airflow / airflow)          | Browser                               |

## Appendix D. Folder layout

---

```
E:\dbt\  
  dbt-env\                                Python virtual environment holding dbt  
  dbt_shaqxe\                             the dbt project  
    dbt_project.yml  
    macros\  
      generate_schema_name.sql  
    models\  
      bronze\sources.yml  
      silver\stg_college_major_roi.sql  
      silver\schema.yml  
      gold\ (18 .sql files and schema.yml)  
    seeds\  
      college_major_roi.csv  
      docs\data_dictionary.csv  
  
E:\airflow\  
  Dockerfile  
  docker-compose.yml  
  .env  
  dags\college_roi_pipeline.py  
  logs\  plugins\  config\  
  
C:\Users\<windows-username>\.dbt\profiles.yml  (credentials, kept outside the project)
```

## Appendix E. Housekeeping

---

Some objects from the early set up may still exist in Snowflake: a bronze copy made before the schema macro, the first silver view, and the two starter models. Check first, then drop what is no longer needed:

### SNOWFLAKE SQL

```
show schemas in database dbt_db;

drop schema if exists dbt_db.dbt_schema_bronze;
drop view if exists dbt_db.dbt_schema.stg_college_major_roi;
drop table if exists dbt_db.dbt_schema.my_first_dbt_model;
drop view if exists dbt_db.dbt_schema.my_second_dbt_model;
```

- Change any password that has been shown in a screenshot, and change the Airflow default login if the machine is reachable by others.
- The Snowflake trial ends 30 days after sign up. Note the date and decide beforehand whether to upgrade or rebuild.
- Without git, copy `E:\dbt\dbt_shaqxe` and `E:\airflow` (leaving out the logs folder) to a safe place from time to time. Never copy `profiles.yml` into a shared location.